

Agent Based Epidemiology Model using INGENIAS Agent Framework Design Team EE-9

Team Members: William R. Moser and Bilal A. Malik

Faculty Advisors: Dr. Li Bai and Dr. Saroj Biswas

Date: April 23, 2010

Abstract

The spread of infection and disease has been a focus of research for societies for centuries. Recent cases have involved the Avian flu, SARS and the H1N1 virus. Governments and businesses spend billions reacting to these outbreaks and attempting to prevent ones in the future. An appropriate computer model can provide a safe and inexpensive way to predict infection propagation and plan actions to mitigate outbreaks. Mathematical models such as the “Susceptible, Infectious, Removed” model (SIR) can provide valuable predictions; however they are derived after the event. The Senior Design team adapted an open-source and general purpose multi agent platform for epidemic modeling, and compared simulation results to an SIR model. The INGENIAS development kit (IDK) from University of Madrid was used to generate agent code for the Java Agent Development Framework (JADE). The Senior Design Team heavily modified this code and developed Applications to create a unique simulation environment running on a standard Windows computer of an epidemic released into a population. A user is able to easily adjust parameters from a graphical user interface and observe outputs comparable to an SIR model using MATLAB. The Senior Design team obtained a SIR model for Hong Kong Flu and compared the MATLAB and Agent simulation results and the resulting plots proved reasonably close.

Table of Contents

I. Background.....	1
II. Literature Review	1
A. Susceptible-Infected-Removed (SIR) Model.....	1
B. Agent Computing.....	3
C. FIPA Agent Standard	4
D. Multi-Agent Platforms	5
E. Past Epidemiology MAS	6
III. Design Objectives	7
A. Functional Description.....	7
B. Project Partitioning.....	8
1. INGENIAS Multi-Agent System Layout.....	8
2. Agent Applications	10
3. Mathematical Model Comparison.....	11
IV. Results.....	11
A. Multi-Agent Simulation	11
1. System Overview	11
2. INGENIAS Multi-Agent System Specification.....	13
3. System Requirements.....	20
B. Analysis of SIR model	21
1. Hong Kong Flu SIR Simulation.....	21
2. Simulation Using Senior Design Project	22
3. Analysis.....	23
V. Conclusions.....	23
VI. Acknowledgments	24
VII. Recommendations for Future Work	24
A. Addition of Dynamic Behavioral Parameters	24
B. Multiple Platform Connectivity	24
VIII. References.....	24
IX. Appendix A – Time Schedule.....	26
X. Appendix B – Budget.....	27
XI. Appendix C – Java Code	28
A. RunSimulation.java.....	28
B. LocAppAppImp.java.....	29
C. CoordAppAppImp.java	32
D. AgentWindowAppImp.....	34
E. AgentDot.java.....	40
F. Sickness.java.....	41
G. CommuteTaskTask.java.....	42
H. GetSickTask.java	43
I. InitializeTaskTask.java.....	45
J. ReportSickTask.java.....	46
K. SendSickLocTaskTask.java	47
L. TestTaskTask.java.....	48
M. WaitForSickTask.java	50

XII. Resumes.....	52
-------------------	----

List of Figures

Figure 1: State Diagram for SIR Population.....	1
Figure 2: Reference Model for Multi-Agent Platform (FIPA00023, 2004)	4
Figure 3: Typical AID of an Agent, “agent-b”	5
Figure 4: Layout of a JADE Multi-Agent Platform.....	6
Figure 5: Screen Shot of the INGENIAS Development Environment	8
Figure 6: INGENIAS Frame Fact Example.....	9
Figure 7: Agent Goal Icons.....	9
Figure 8: Agent Task Icons.....	9
Figure 9: INGENIAS Interaction Model Using The GRASIA Specification.....	10
Figure 10: INGENIAS Agent Definition.....	10
Figure 11: NetBeans Java Development Environment.....	11
Figure 12: Epidemic Simulation System Behavior.....	12
Figure 13: Citizen Agent Diagram.....	13
Figure 14: Citizen Task Diagram.....	14
Figure 15: Threading Background Actions.....	15
Figure 16: Coordinator Agent Diagram.....	16
Figure 17: Coordinator Agent Task Diagram	17
Figure 18: Coordinator Running an Application Window	17
Figure 19: Starting an INGENIAS JADE Agent	18
Figure 20: User Input and Agent Deploy Frame	18
Figure 21: Agent Movement and Status Graphical Output	19
Figure 22: Simulation Sub Populations and History Logging.....	19
Figure 23: Application Front End Final Product	20
Figure 24: JADE Agent Management GUI	20
Figure 25: MATLAB Simulation of SIR Epidemiology Model.....	22
Figure 26: Senior Design Team Simulation Results.....	23

List of Tables

Table 1: Project schedule.....	26
Table 2: Project Budget.	27

I. Background

The spread of infection and disease has been a focus of research for societies for centuries. Recent cases receiving international attention have involved the Avian flu, SARS and the H1N1 virus. The health and monetary cost of these illnesses are immense. By the end of 2009, the United States government spent over 6.15 billion dollars on H1N1 “swine flu” vaccinations. Computer modeling can provide a safe and inexpensive way to predict infection propagation and plan actions to mitigate outbreaks. The classic Susceptible-Infectious-Removed (SIR) differential model was famously used by Louis Pasteur to study small pox. A more recent development in modeling comes from multi-agent computing. This breaks down the population into many individual people, programmed as agents. Contrary to the top-down approach of the differential model, which models a system as a whole, a multi-agent simulation goes bottom-up and develops the system behavior from the interactions of all the agents within the system. This method extends naturally into epidemiology as the spread of many diseases is dependent on human actions. Incorporating a graphical user interface (GUI), our approach will allow health officials with little programming experience to finely tune a simulation to match the interaction of a disease with a single individual, and, thanks to the dynamic behavior of agents, be able to reliably scale the population to model an outbreak. At the same time the model should provide some insight into the behavior of individuals during these events, something not possible with a system level model.

II. Literature Review

A. Susceptible-Infected-Removed (SIR) Model

The SIR model is mathematically represented as a set of differential equations that model a population’s transition between different stages of an epidemic (Tassier, 2005) . These stages of an epidemic can be categorized into three distinct states: Susceptible, Infectious, and Removed, and comprises the state machine shown in Figure 1. The Susceptible state represents individuals capable of becoming infected with a virus. The Infectious state represents individual exposed to the virus by which they experience symptoms. In addition they are also capable of infecting other people. Finally, the Removed state represents individuals that are unable to contract the disease. This includes individuals that have recovered from the illness and have developed a resistance to future infections. Collectively these three states effectively form a Markov Chain, in which the conditional probability distribution of the future state depends only upon the present state.

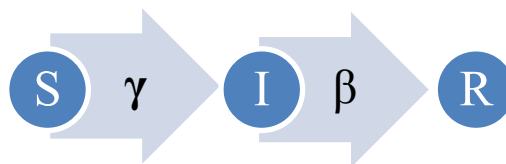


Figure 1: State Diagram for SIR Population

As outlined in Olinick, The SIR model describes the transition between individuals from one state to another, tabulated in terms of gains and losses. For example, if two agents transition from the Susceptible to the Infectious state, then the Susceptible state loses two agents, while the Infectious gains two agents. If $Y(t)$ represents the number of individuals in one state at time t , then Gain-Loss represents the net change in the size of that state.

$$\frac{dY}{dt} = \text{Gain} - \text{Loss}$$

Using this methodology, differential equations can be derived to model the transition between each state as shown below.

$$\frac{dS}{dt} = -\text{Loss in } S$$

$$\frac{dI}{dt} = \text{Gain in } I - \text{Loss in } I$$

$$\frac{dR}{dt} = \text{Gain in } R$$

It is important to note that the gain for the Susceptible state and loss for the Removed state cannot possibly exist. For example, an individual cannot become Susceptible if currently in another state, and once an individual is removed it cannot leave that state.

Another factor which affects the model is the concept of a sub-population within each state. When a susceptible individual comes within reasonable proximity of an infected individual it can be assumed that the individual will become infected. However, not all of the susceptible individuals will be within a reasonable distance of infected individuals. Therefore, probability becomes an important factor in spread of diseases. For example, if there is a large number of susceptible individuals then the probability of an interaction increases, which in turn increases the spread of disease. In addition, given that not all interactions result in transmission of the disease, the variable γ can be used to represent the probability that an interaction occurs and that the disease is transmitted. In order to make the differential equations more dynamic and realistic, the rate of recovery β , and the rate of transmission γ are incorporated into the differential equations. The rate of recovery can be estimated as $\beta = \frac{1}{k}$, in which k represents the infectious period.

$$\frac{dS}{dt} = -\gamma S \cdot I$$

$$\frac{dI}{dt} = \gamma S \cdot I - \beta \cdot I$$

$$\frac{dR}{dt} = \beta S \cdot I$$

From a mathematical analysis $S > \frac{\beta}{\gamma}$ which can be derived given the condition $(\gamma S \cdot I - \beta \cdot I) > 0$ and given that S and I are always positive numbers. Essentially, whenever the number of susceptible is less than the ratio $\frac{\beta}{\gamma}$, the number of infected individuals will decrease. This is the most fundamental relationship from the SIR model. Therefore the

most vital parameters are S_0 , β and γ , and in order to reduce the spread of the disease $S < \frac{\beta}{\gamma}$. (Olinick, 1978)

Another example of a similar epidemiology model is the Susceptible, Infectious and Susceptible (SIS) model. In this configuration, the population does not develop a resistance to disease and therefore the system oscillates between two states. A very simple example of the SIS model is the common cold. The common cold does not give lengthy permanent resistance (Dunham, 2005). The SEIR model adds an intermediate state to the SIR model, where an individual has been exposed to a virus and is contagious, but is not experiencing any symptoms. As the virus takes effect, the individual moves into the Infected state. The SEIR have been used for epidemics like the Lassa virus in Lassa, West Africa, which is a blood draining fever virus with a high death rate. The Lassa virus is carried by rats and is also known to be a possible bio-warfare weapon. The time between being exposed to the virus and showing symptoms can range from six to twenty one days (Dunham, 2005).

Although SIR and related models are reasonably accurate epidemiology models, such top-down systems have weaknesses. They rely on large averages, such as a large homogenous population that moves uniformly and gets sick at the same rate. Additionally, SIR is a linear time invariant (LTI) system. Modeling becomes significantly more complex if time-varying elements are introduced. In epidemiology, an example of this would be a strain of bacteria developing a resistance to antibiotics.

B. Agent Computing

The definition of an agent is not standardized in computing. For the purposes of this project, we will adopt the Institute for Electrical and Electronic Engineers (IEEE) definition of an agent, as described in Gerhard Weiss's *Multi-Agent Systems*. Weiss defines an agent as a computing system capable of autonomous action in an environment in order to meet its design objectives. The key feature of multi-agent systems (MAS) is that all of the agents have control over their own actions.

Agents exist mainly as software entities, utilizing object-oriented programming languages such as Java (IEEE Foundation for Intelligent Physical Agents, 2010). Java objects are also software elements within a program and share many characteristics with agents, such as an inherited structure and unique tasks, or methods (Oracle Corporation, 2010). Multi-agent programming differs from Java's object oriented programming in that it demands autonomy. With objects, a master class can simply call a method of an object and obtain a result. In many cases this is desirable, such as a payroll program. An agent has its own individual goal to achieve, and will only cooperate with a request from another agent if it is in line with its goals. This type of software is ideal for systems that do not communicate often with a master control, such as a deep space satellite, where it would be more practical to make decisions on board and initiate a response without external control (Weiss, 1999). As illustrated in that example, an agent will typically sense its environment (satellite power, light) and will have a list of possible actions that can be executed to modify the environment. This type of scenario also introduces the idea of an intelligent agent. Intelligence, a general term, is restated as agent flexibility (Weiss, 1999), requiring that a flexible agent is:

1. Reactive – Agents can sense their environment and act to achieve their design objectives
2. Proactive – Agents have goal-directed behavior and initiate actions achieve their design objectives.
3. Social – Agents are able to interact and communicate with other agents in order to meet their design objectives (Weiss, 1999).

This behavior is facilitated by a multi-agent platform, a set of libraries and software tools that allow programmers to create and manage software agents (IEEE Foundation for Intelligent Physical Agents, 2010). The specific structure and function of multi-agent platforms has been made an IEEE standard and will be discussed below.

C. FIPA Agent Standard

Multi-agent computing is a relatively recent field of research. To aid the development of such systems, the Foundation for Intelligent Physical Agents (FIPA) was formed in Switzerland in 1996 to produce software standards for interactions in agent-based systems. Since then they have been adopted by the IEEE Computer Society and are now a widely recognized set of standards. These standards are split across several aspects of agent software: agent communication, agent transport, agent management, abstract architecture and Applications (IEEE Foundation for Intelligent Physical Agents, 2010). Abstract agent platform architecture was taken from FIPA Standard SC00023 and is shown in Figure 2.

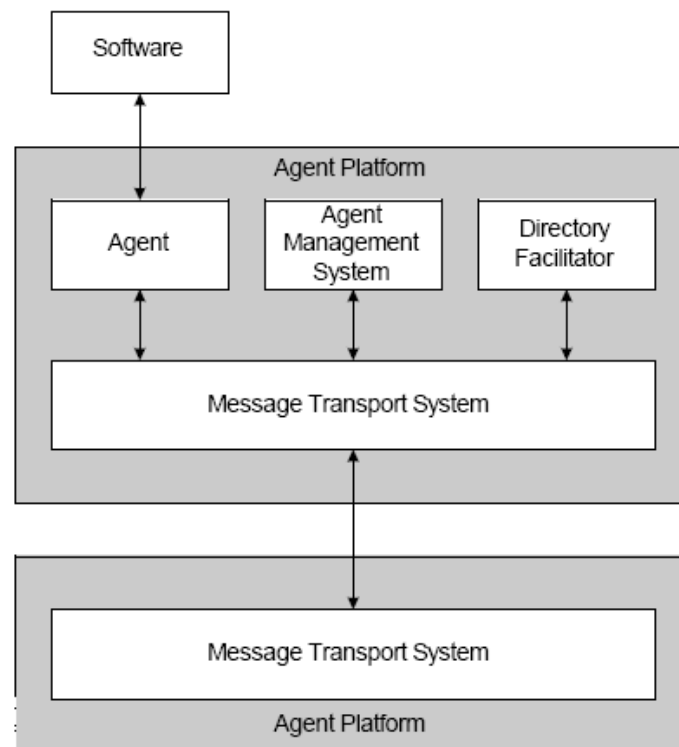


Figure 2: Reference Model for Multi-Agent Platform (FIPA00023, 2004)

In this model, FIPA defines agents as computational processes that implement autonomous action and communication within an Application. Agents communicate

using the Agent Communication Language (FIPA-ACL), which will be illustrated later. The Directory Facilitator (DF) acts as a “yellow pages” for the agent environment. It contains the Agent Identifier (AID), a unique name and address (a URL), as a way to contact the agent as well as the types of actions and services associated with that agent. An example of this notation is shown in Figure 3.

```
(agent-identifier
 :name agent-b@bar.com
 :resolvers (sequence
  (agent-identifier
   :name ams@foo.com
   :addresses (sequence iiop://foo.com/acc))))
```

Figure 3: Typical AID of an Agent, “agent-b”

The Agent Management System (AMS) is a critical part in a FIPA multi-agent platform. Although agents act autonomously, they need an overarching supervisor. The AMS performs such tasks as creating and destroying agents and registering AIDs. Where the DF provides yellow pages for finding agent services, the AMS provides the “white pages” - an ordered listing of all agent identifiers. The most important aspect in the multi-agent platform is the Message Transport System (MTS), which handles messaging between all elements besides direct software agent query using the Agent Communication Language (FIPA-ACL). Key phrases in this language are “understood”, “not understood” and “reply-with” (FIPA00067, 2002). As one can see, they model interactions between elements and agents much like those of humans. The content of the messages sent are specific to the particular agent platform. The MTS stores all these messages in a cache and distributes them to their intended recipient. Once an agent or element receives a message they respond with the ACL message “received” and the message is cleared from the MTS cache. As shown in Figure 2, a FIPA agent platform's MTS can send messages out of the platform and to another platform's MTS. This allows for agent computing over multiple machines and devices, using any type of multi-agent platform that complies with the messaging standard.

D. Multi-Agent Platforms

Several multi-agent platforms exist that comply with the FIPA standards.

ZEUS, an Open Source agent system entirely implemented in Java by BT Labs, is a toolkit for constructing collaborative multi-agent Applications (Nwana, Ndumu, Lee, & Collis, 2002). Zeus provides a graphical user interface in JSwing to represent agent behavior. Zeus uses the FIPA ACL as the message transport and TCP/IP sockets as the delivery mechanism for inter-agent communication. Again, the Zeus approach to design the agent interactions is laid out graphically. They form a decision flowchart that moves agents to different actions and states if certain preconditions are met. This platform provides an organized view of could otherwise be a complex scheme. The platform is tested to work on Windows 95/98/NT4 and Sun Solaris operating systems (IEEE Foundation for Intelligent Physical Agents, 2010).

JADE, developed by TILAB in Italy is another set of tools, implemented in Java, to simplify the development of multi-agent Applications (Telecom Italia SpA, 2000). It provides a GUI for the AMS to monitor the life cycle of all agents deployed. It also facilitates debugging with sniffer agent and dummy agent creation. The JADE agent platform also allows for agent deployment over multiple computers. JADE messaging is handled locally using Java Remote Method Invocation, while messaging to other devices is handled using the IIOP protocol. A JADE agent platform can be distributed over several machines. Agents can also migrate or clone themselves to other hosts of the platform, regardless of the OS. The life cycle of agents can be remotely controlled via a GUI, which also allows debugging tools to be started. JADE is implemented in version 1.2 of JAVA and has no further dependency on third-party software (Telecom Italia SpA, 2000).

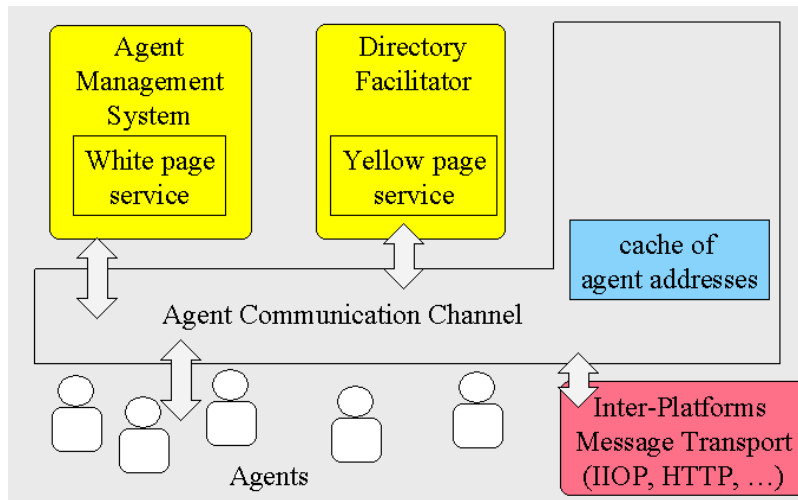


Figure 4: Layout of a JADE Multi-Agent Platform

E. Past Epidemiology MAS

Past epidemiology multi-agent models have worked generally well. Jill Bigley Dunham of George Mason University demonstrated how their in-house developed MASON multi-agent libraries and development kit could be used to simulate SIR, SEIR and SIS type infections (Dunham, 2005). The scope of this project spans years, as the platform libraries were developed by George Mason University and the social structures well defined by social science. The output states of all the agents were summed up and plotted over the simulation time in order to compare the model to classical SIR models. This was an extremely successful simulation given the level of agent complexity was low. Its in-house design, however, makes interoperability a concern. The model used (Teweldemedhin, Marwala, & Mueller, 2004) achieved a similar end using JADE. This was specific to the case of the HIV virus.

III. Design Objectives

A. Functional Description

The senior design team developed a multi-agent simulation of humans experiencing an epidemic to run on a standard Windows personal computer. The simulation was written in Java and developed using open source software tools and complies with the IEEE FIPA standard for agent communication. The simulation results were to be compared to those of an existing proven epidemiology model.

The deliverable of this project is the multi-agent simulation software Application, which includes a graphical user interface (GUI) for simulation control as well as output and run-time observation. The GUI as well as other Java Applications run locally on individual agents. A user is able to input basic disease parameters which are passed to independent software agents before they are deployed. The agents exhibit some basic human behavior and are represented graphically as dots in a two dimensional space. There they can be seen moving, contracting a disease and passing the disease to other agents. The population of human agents is polled regularly to determine their health status. Sub-populations based on health status are tallied and results saved to a file to be plotted for comparison against existing epidemiological models. MATLAB was used to plot the simulation results as well as the existing mathematical model.

Requirements for the deliverable dictate that the simulation run on a standard personal computer running Windows XP. A PC donated to the project by the Computer Fusion laboratory served as the minimum system requirements, utilizing an Intel Core 2 Duo processor operating at 3.0 GHz with 3 GB of RAM. The agents created for the simulation were required to comply with the IEEE FIPA standard for multi-agent systems. The entire project was to be developed using only (aside from Windows XP) open source tools to produce a reusable, reconfigurable, freely distributable product. Finally the output of the simulation was required to be a plot in the same form as an existing mathematical model and produce results close to a real world disease case study.

The JADE agent platform was chosen for agent management, as it was both open source and FIPA compliant and ran on the Windows XP/Vista operating systems available in our advisor's laboratory. The agents are also coded in Java, for which there was some technical support available from the Computer Fusion laboratory. The INGENIAS development kit (IDK) was chosen to model the multi-agent system and generate some Java code for the agent simulation. The IDK is a code generator that allows a developer to layout a multi-agent system graphically. It was developed and is still supported in version 2.8 by the Universidad Complutense Madrid (UCM). An agent structure and a communication protocol, compliant with the IEEE standards, are defined from this layout. The developer provides Java code for tasks and Applications for the agents to execute. The model chosen for benchmarking was the SIR model, a set of differential equations discussed earlier in this paper. The disease chosen was an outbreak of Hong Kong Flu in 1968.

B. Project Partitioning

The design of this project was partitioned into three major software components: an IDK layout, Java Applications, and SIR modeling. The INGENIAS development kit was used to graphically lay out and define two types of agents, a citizen agent and a coordinator, and the communication between them. Java Applications were developed separately for the agents to utilize during the simulation, including a GUI. Finally, a mathematical model was researched and demonstrated using MATLAB.

1. INGENIAS Multi-Agent System Layout

Due to the scope of this project and time constraints, a code generator was used to define the more basic aspects of agent behavior. The INGENIAS development kit provides an expansive library of agent characteristics represented with graphical icons that can be laid out and connected to form a Specification. Figure 5 depicts the development environment. The top-left window contains the diagram types. These include Agent Definitions, Task and Goal Diagrams, and Interaction Models. Below this is a window containing all the icons used in the current MAS layout. Each icon is associated with a separate Java file in the project folder. Immediately to the right of that is a tool bar containing all the icons allowed to be placed in the current diagram type. Clicking any of the icons adds it to the model and a file is created upon the next code generation, triggered by clicking the “INGENIAS Agent Framework generator” or from the command prompt. Between generations, the layout is saved as an XML file, containing the visual layout of all diagrams, displayed in the main window.

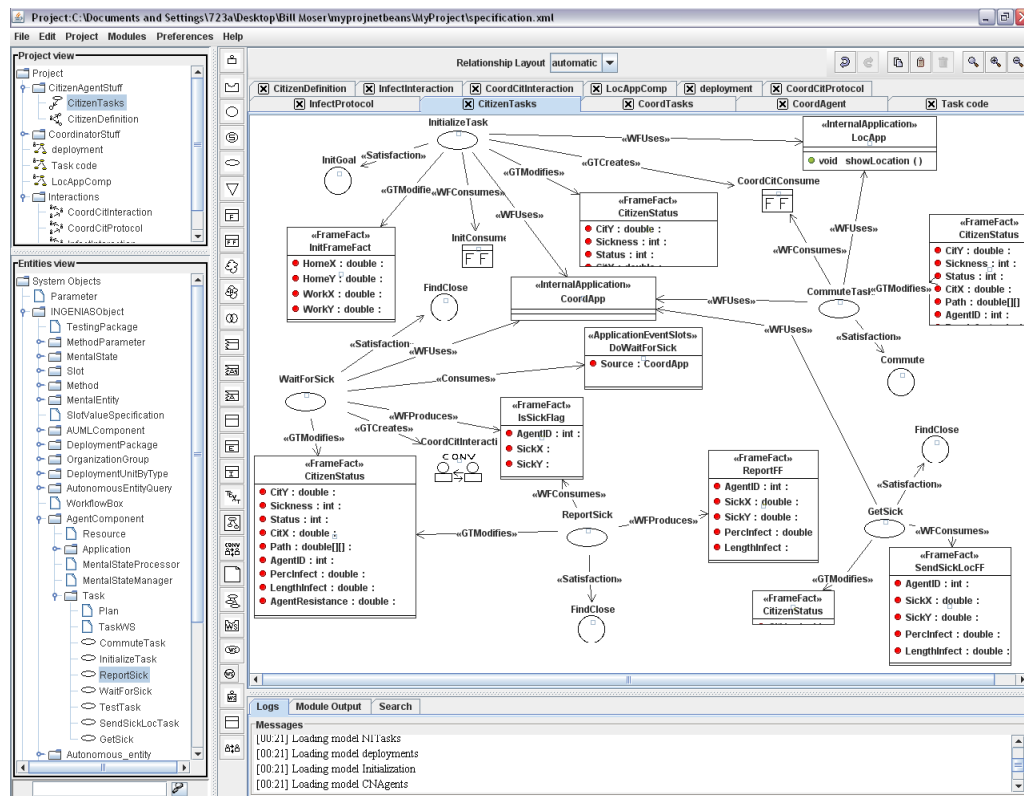


Figure 5: Screen Shot of the INGENIAS Development Environment

Only a few of the INGENIAS code structures were used for this project. The Frame Fact is a Java class used to store information, as shown in Figure 6. The only methods provided are “set” and “get” methods to retrieve information, but no processing occurs. This object is stored in the Agent Mental State, a vector within the JADE agent.

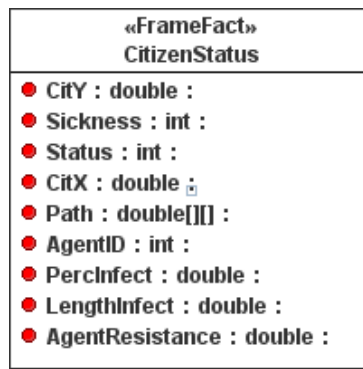


Figure 6: INGENIAS Frame Fact Example

Goals, depicted in **Error! Reference source not found.**, are stored in the Agent Mental State in a similar style. Agents take actions based on programmed goals, which can be created at run time as well, to reflect changing priorities.



Figure 7: Agent Goal Icons

Tasks, depicted in Figure 8, are the actions for an agent to execute. Most often these refer to outside Applications, or a few lines of code can execute from within the Task structure. Tasks are executed one at a time, that is, a Task will not begin execution until the last is finished.



Figure 8: Agent Task Icons

INGENIAS Conversations are defined using the GRASIA Specification for interactions. This is a defined protocol based on ACL messaging between the JADE agents. Conversations contain an Initiator, which launches the conversation from a Task or Application, and Collaborators, which can be a single agent or multiple agents of a single type. Conversations use the JADE Yellow Pages to find the correct agent or agent type to be messaged. Once the appropriate parties have been connected, Interaction Units define intermediate steps for messaging and order them in time, e.g. First: Send Invitation, Second: Send Accept Invitation.

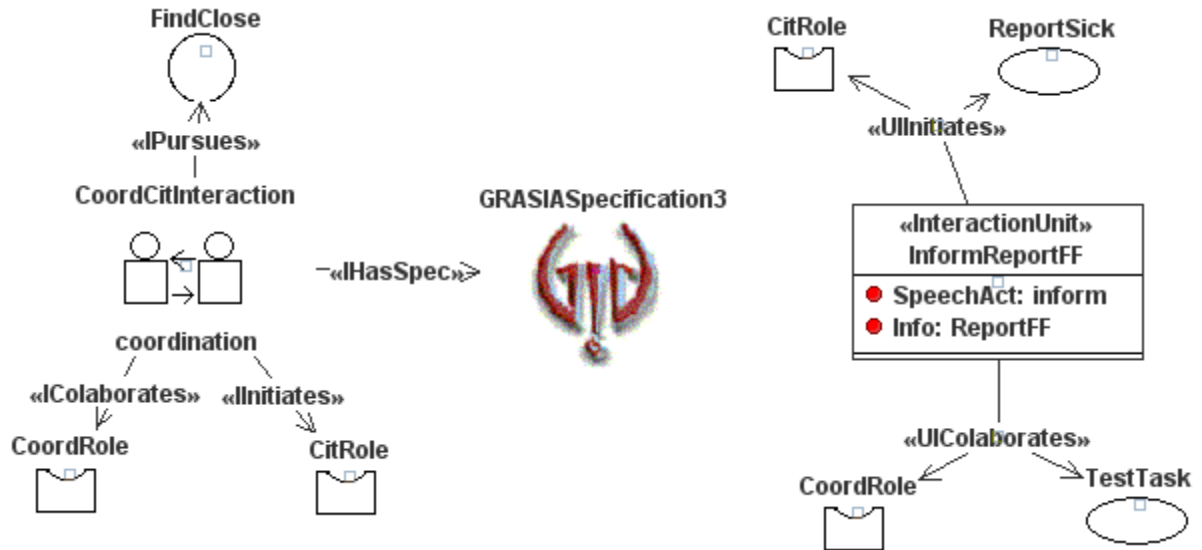


Figure 9: INGENIAS Interaction Model Using The GRASIA Specification

The most crucial structure was the INGENIAS JADE Agent, depicted in Figure 10. These agents associate all the Goals, Tasks, Frame Facts, interactions and Applications at startup by including them in vectors within the agent. The agent is responsible for scheduling the Conversations and Tasks sequentially to be executed. The Conversations and Tasks are executed when all specified inputs are present. Applications are also associated with an icon, although this is merely a placeholder and only class headings are generated.

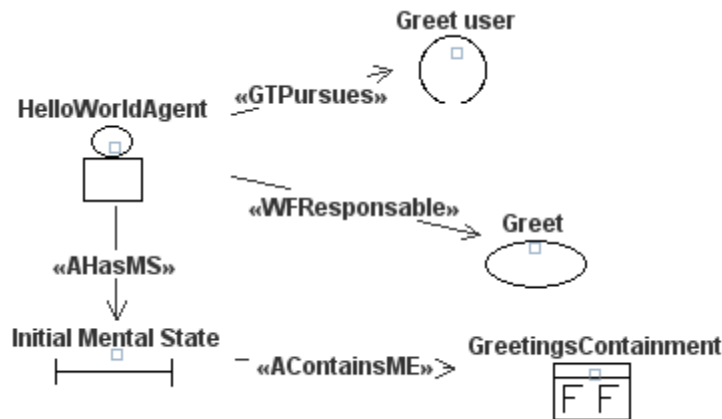


Figure 10: INGENIAS Agent Definition

2. Agent Applications

INGENIAS Tasks only allow for sequential execution, so any action running in conjunction with other Tasks or continuously during the simulation needed to be developed separately, associated as Applications in INGENIAS, and then threaded. These Applications were developed using the NetBeans integrated development environment (IDE), shown below in Figure 11. Applications developed for this simulation include a GUI, a disease timer, and a pathing Application.

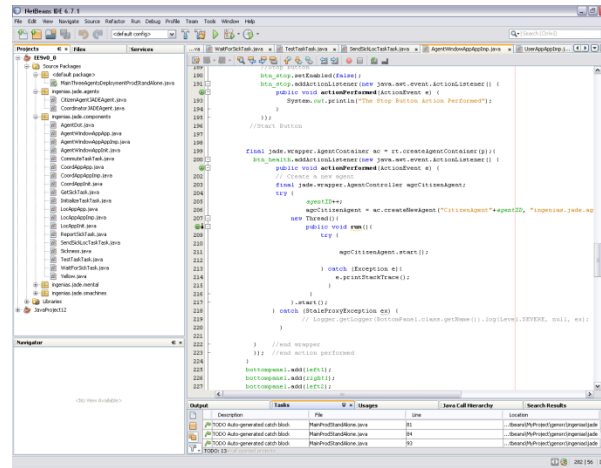


Figure 11: NetBeans Java Development Environment

3. Mathematical Model Comparison

In order to measure the success of the multi-agent simulation, it was benchmarked against an existing mathematical epidemiology model. A case study attempting to model a real world outbreak using one of these models and the resulting equations were sourced. MATLAB, from the Mathworks, was used to solve one of these differential models and plot the results for comparison.

IV. Results

A. Multi-Agent Simulation

1. System Overview

The multi-agent simulation Application was successfully developed using the aforementioned tools and executed on a Windows computer. The agents designed for this project were a Coordinator agent and a Citizen agent. The interaction between them at a system level is illustrated in Figure 12. The simulation is started by deploying a number of susceptible agents into the simulation environment. Then, at least one sick agent is deployed and begins to infect the other agents. The sick agent, when deployed, waits a short amount of time to avoid error then sends an ACL message to the Coordinator agent containing the agent's current location as Cartesian coordinates, as well as the length of the infection period and percent transmission chance for the disease currently afflicting the agent. The Coordinator receives the message, checks to ensure the agent was indeed infected, and then sends a copy of the message to all Citizens agents. The Citizen agent receives the message and checks to ensure that it is susceptible. If susceptible, the agent generates a random double precision number between zero and one. If this number exceeds the percent infection chance from the ACL message, the agent then checks to see if the position of the infected agent is within range. If the two agents are sufficiently close, the length of infection is applied to the susceptible agent, reduced by the agent's resistance level – a value ranging from 0.3 to 0.99. The now sick agent counts down this time, representing hours, while attempting to infect other agents. This behavior is designed to test parameters sequentially, reducing the amount of computation performed

by agents, as well as reduce messaging, by including as much information as possible in a single ACL message.

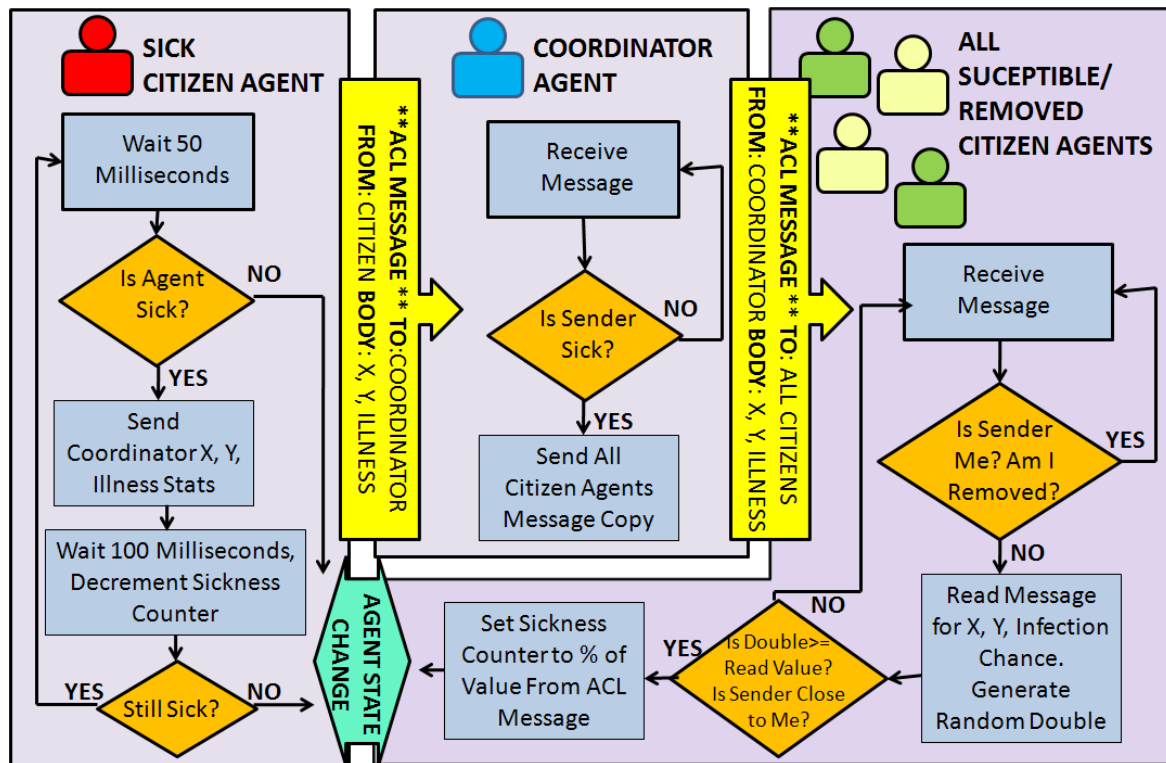


Figure 12: Epidemic Simulation System Behavior

2. INGENIAS Multi-Agent System Specification

a) Citizen Agent Definition

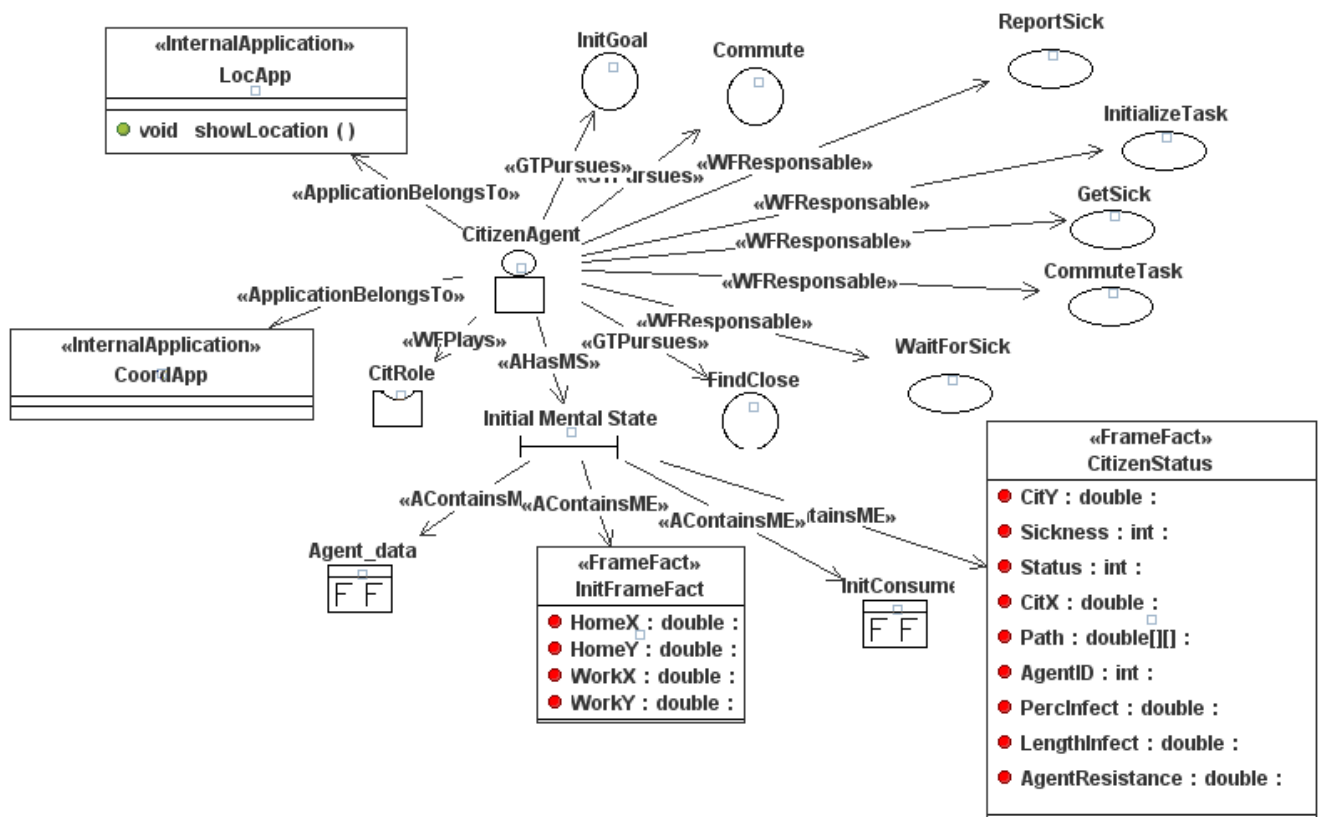


Figure 13: Citizen Agent Diagram

The Citizen Agent definition diagram is shown above in Figure 13. The Citizen agent performs five types of tasks, pursues three goals, accesses two Applications, and retains some information in two Frame Facts. These are described in further detail below.

b) Citizen Agent Tasks

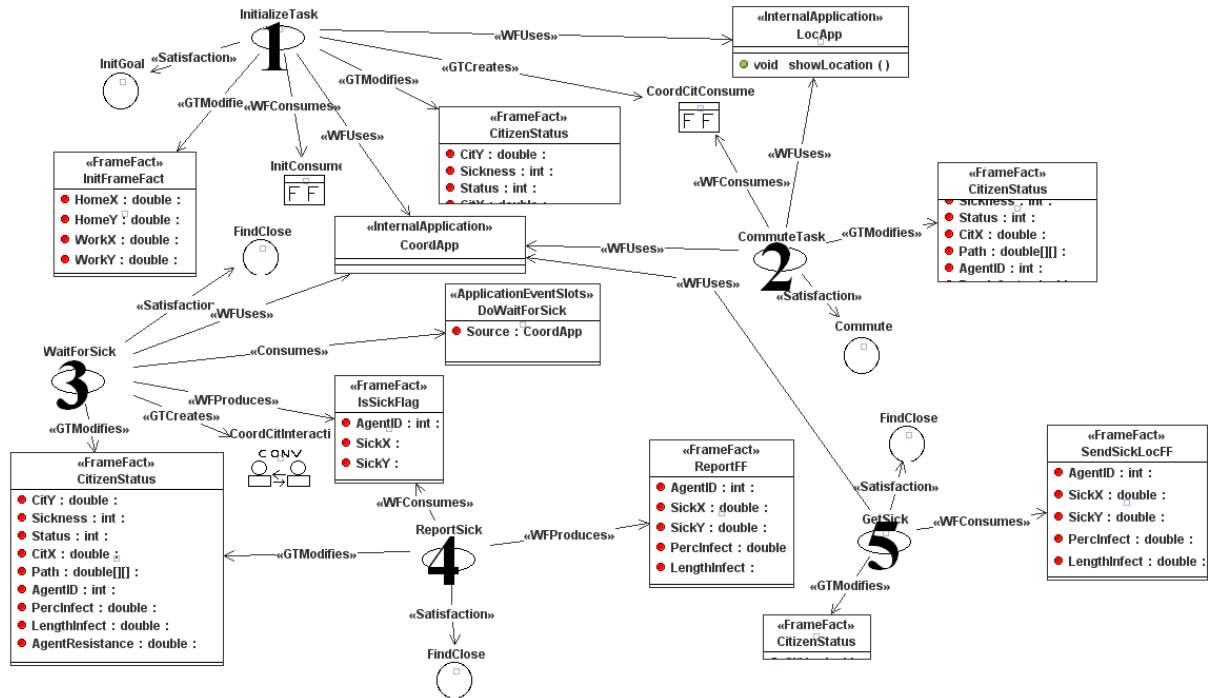


Figure 14: Citizen Task Diagram

The Tasks layout for the Citizen agent is shown above in Figure 14, with the Tasks numbered. InitializeTask (1), takes in the percent infection chance and length time infected from the user and stores it in a “CitizenStatus” Frame Fact along with an Agent Identification Number, starting from one (Agent1). Since the disease is stored in each agent at deployment, disease parameters can be adjusted to allow for multiple infections within one simulation. InitializeTask also accesses an Application, LocApp, to produce a random home and work location for the agent to travel between, as well as the path in between. These values are stored in CitizenStatus. After completion, the Task produces a dummy object, CoordCitConsume. The next task, CommuteTask (2), will not execute until this object exists. This is to ensure the intended order of execution, as Tasks are typically scheduled randomly according to the INGENIAS framework. After CommuteTask takes the dummy input, it accesses the Application, CoordApp, to start the agent moving between work and home. This is the extent of our human behavior. The majority of individuals in a society have jobs to which they commute daily, so the senior design group has modeled agents to travel back and forth between the two as well. If the agent is deployed as a sick agent, CoordApp produces an event “DoWaitForSick” which is the input necessary to begin WaitForSick (3). This task retrieves the current location of the sick agent, along with its agent ID, and launches a conversation to send it to the Coordinator agent. The conversation diagram is shown earlier in Figure 6. The ReportSick Task (4) sends the agent ID, current location, percent infection chance for the disease and the length of the infection to the Coordinator agent. If the Citizen agent is susceptible, the GetSick Task (5) infects the agent using the CoordApp Application when it receives a message from the coordinator agent, shown as a Frame Fact, “SendSickLocFF.”

c) Citizen Agent Applications

The INGENIAS code generator creates a class heading in Java with whatever abstract methods the designer believes he or she will need. The designer then creates another class to overwrite these abstract methods with actual code. The Citizen agent is associated with two Applications, which are instantiated once for every Citizen agent deployed so that all public methods and variables associated with the Application are unique to the instance associated with the agent. LocApp was developed first and provides some utilities for the agent. The method agentMove() returns a random set of Cartesian coordinates for the agents' home and work locations within the maximum size of the simulation environment. It then interpolates a path between the two with a random number of intermediate points ranging from thirty to fifty. LocApp also provides a method that calls another class, Sickness. If an agent is scheduled to be infected, the getSick() method is called. This takes the length of the infection period and a resistance level and starts a sickness counter. The counter decrements the reduced infection period each time step, to represent a human recovering from a disease. This method must run continuously during the simulation, so it was threaded using Java's Thread.start() method, shown in Figure 15. The counter counts down in milliseconds using the Thread.sleep() method and then stops counting with the sickness counter reaches zero with Thread.yield().

```
new Thread() {
    public void run() {
        try {

            countDown(Sickness1, Resistance1);
        } catch (InterruptedException ex) {
            Logger.getLogger(Sickness.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}.start();
```

Figure 15: Threading Background Actions

The second Application used by Citizen agents is CoordApp. CoordApp's main function is to move the agent, each time step to the next location in its path array. This is another threaded method, as the agents move back and forth continuously. If during its movement, it becomes infected, it produces the event DoWaitForSick, discussed earlier. CoordApp also provides a method to test if another agent is close to its current position. This distance checking uses the Pythagorean Theorem to find the distance between two points and returns a Boolean value indicating whether the two points are within a threshold, which was thirty pixels for our simulation. This method of determining if multiple entities are close to each other distributes the computational burden across all agents and is used in video game design.

d) Coordinator Agent Definition

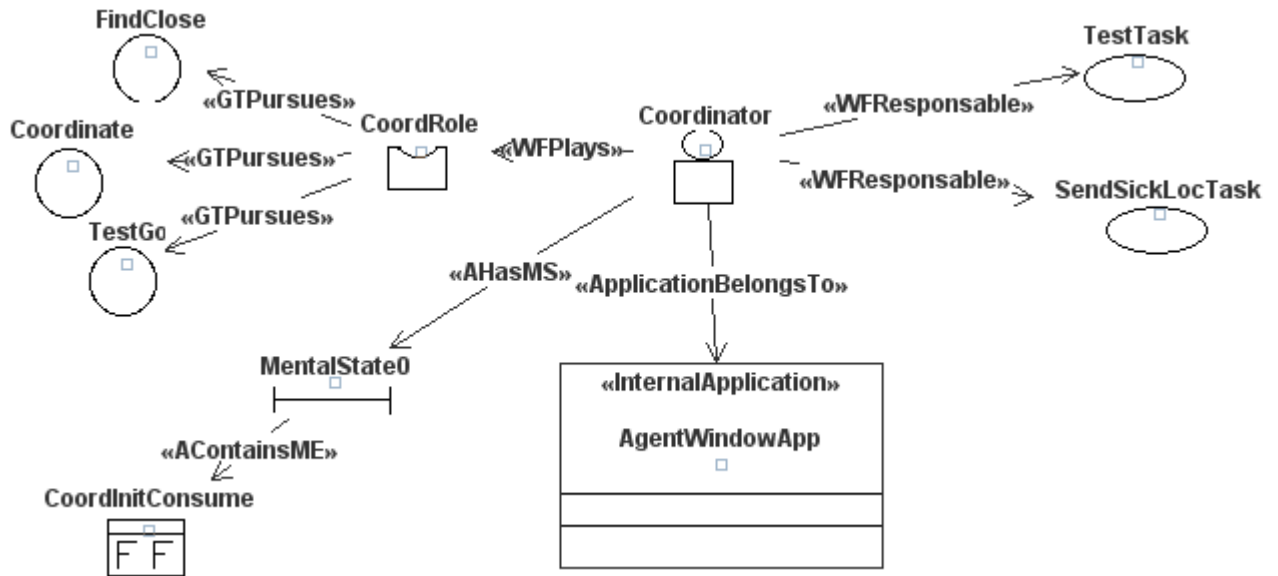


Figure 16: Coordinator Agent Diagram

The Coordinator Agent definition diagram is shown above in Figure 16. The Coordinator agent performs two types of tasks, pursues three goals, and accesses one Application. These are described in further detail below.

e) Coordinator Agent Tasks

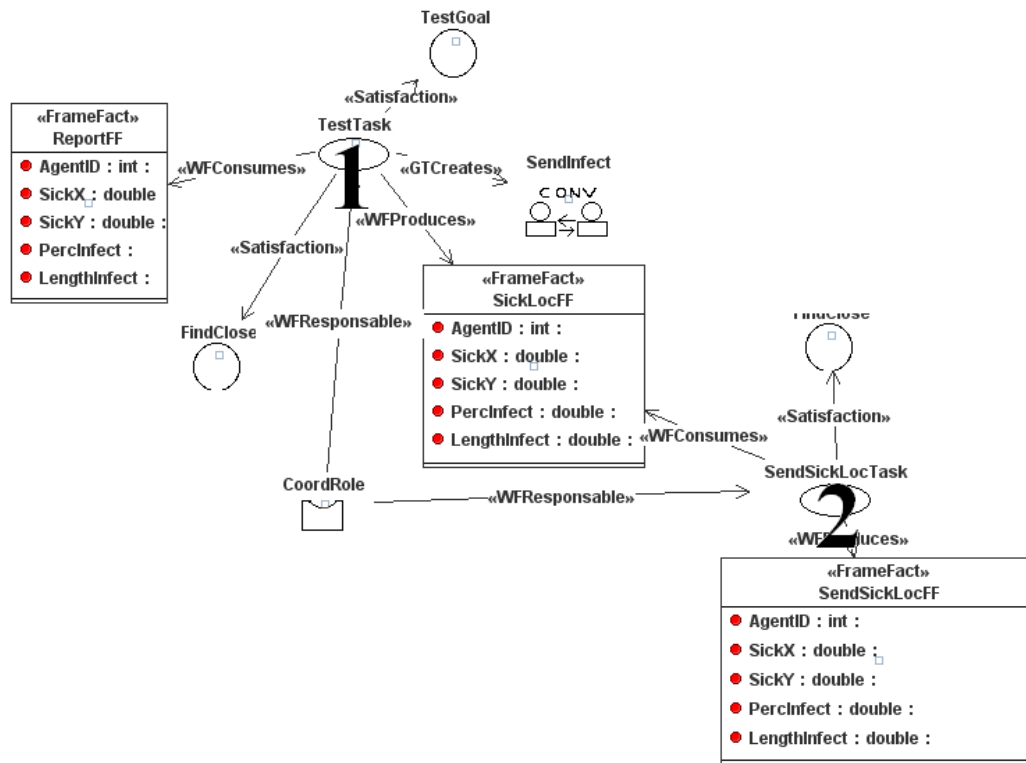


Figure 17: Coordinator Agent Task Diagram

The Tasks layout for the Coordinator agent is shown above in Figure 17, with the Tasks numbered. TestTask (1) waits to receive the Frame Fact “ReportFF” from an ACL message from a sick Citizen agent. When the Coordinator receives the message, it launches a conversation with all Citizen agents. The Task, SendSickLocTask (2) sends a copy of the message, “SendSickLocFF” containing the sick agent’s ID, location and infection parameters out to all agents for them to process. These Tasks should be executed sequentially, so their behavior was coded within the INGENIAS generated Task code.

f) Coordinator Agent Application

The Coordinator accesses one complex Application, called AgentWindowApp. This Application is the entire front-end graphical user interface (GUI) for the simulation. Even if multiple Coordinators are deployed to distribute messaging tasks, they still access a single instance of AgentWindowApp within the original Coordinator. This is indicated to the user by including the name of the agent running the Application in the window heading, as shown in Figure 18.



Figure 18: Coordinator Running an Application Window

The GUI includes space for inputs from a user for the percent chance that the infection is passed to an agent as well as the length of time the agent is infected and whether the

agent is deployed as infectious or susceptible. The agent is then deployed using a “Deploy” button. The Senior Design team used the method for starting JADE agents automatically generated by the IDK and applied it to our button behavior, shown in Figure 19. This method prevented deployment of more than one agent at a time, so a user must click the button multiple times to deploy multiple agents. The input frame for the GUI is shown in Figure 20.

```
// Create a new agent
final jade.wrapper.AgentController agcCitizenAgent;
try {
    agentID++;
    agcCitizenAgent = ac.createNewAgent("CitizenAgent"+agentID, "ingenias.jade.agents.CitizenAgentJADEAgent", new Object[0]);
    new Thread(){
        public void run(){
            try {
                agcCitizenAgent.start();
            } catch (Exception e){
                e.printStackTrace();
            }
        }
    }.start();
}
```

Figure 19: Starting an INGENIAS JADE Agent



The image shows a dark red rectangular frame containing a user input interface. It includes two text input fields: "Infection Time in Hours" with the value "20" and "% Infection Chance" with the value "15". To the right of these fields are two radio buttons: "Sick Agent" (which is selected) and "Healthy Agent". A "Deploy" button is located on the far right of the frame.

Figure 20: User Input and Agent Deploy Frame

The center portion of the GUI, shown in Figure 21, renders the agents as colored dots in a two-dimensional space, representing a city or town. The area is presently set to 600 by 600 pixels, but is certainly expandable, if needed. Agents are drawn every fifty milliseconds to ensure that the visual is true to the simulation, which is running at least two orders of magnitude slower. Agents pass their location in the space along with their current health status, which is 1 for susceptible, 2 for infectious and 3 for removed to the GUI. The dot is then drawn in the proper location as green, yellow or red to reflect the status, respectively. This provides a user with simulation-time overview of how the infection spreads geographically through the population. Such information is often useful for epidemiologists.

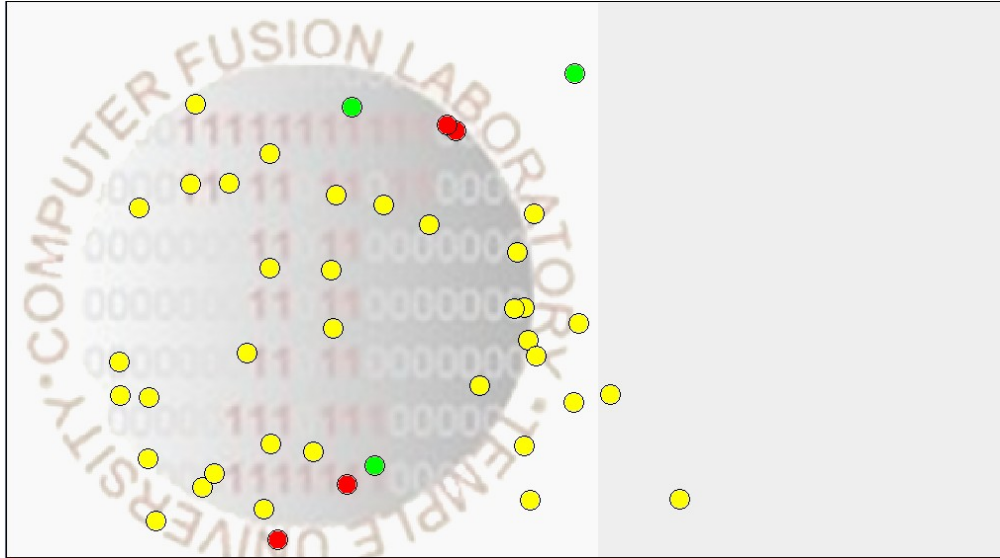


Figure 21: Agent Movement and Status Graphical Output

The top portion of the GUI, shown in Figure 22, contains the only simulation-time statistics for the Application. Every fifty milliseconds, the colors of all the dots are tallied and displayed numerically to show the number of Citizen agents currently susceptible, infectious or removed. These values are stored in a hash table within the Application every time-step. To the right of the population count is the “Save History” button, which flushes the hash table to a text file. This text file contains the three sub populations in three columns and can be plotted using a plotting tool such as Microsoft Excel or The Mathworks’ MATLAB.

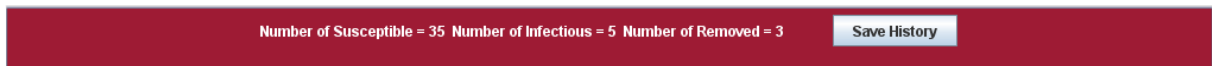


Figure 22: Simulation Sub Populations and History Logging

As shown in Figure 23, the finished GUI serves to visually accompany the user’s understanding of the agent behavior. Aside from the developed GUI, the JADE agent management window is also launched, shown in Figure 24, so that users may monitor whether agents are starting successfully and are active. Providing the JADE window also allows users to set up a JADE remote agent platform, to run agents across multiple computers. While this feature was not employed by the senior design team, the window was left for technical demonstration and future expansion.

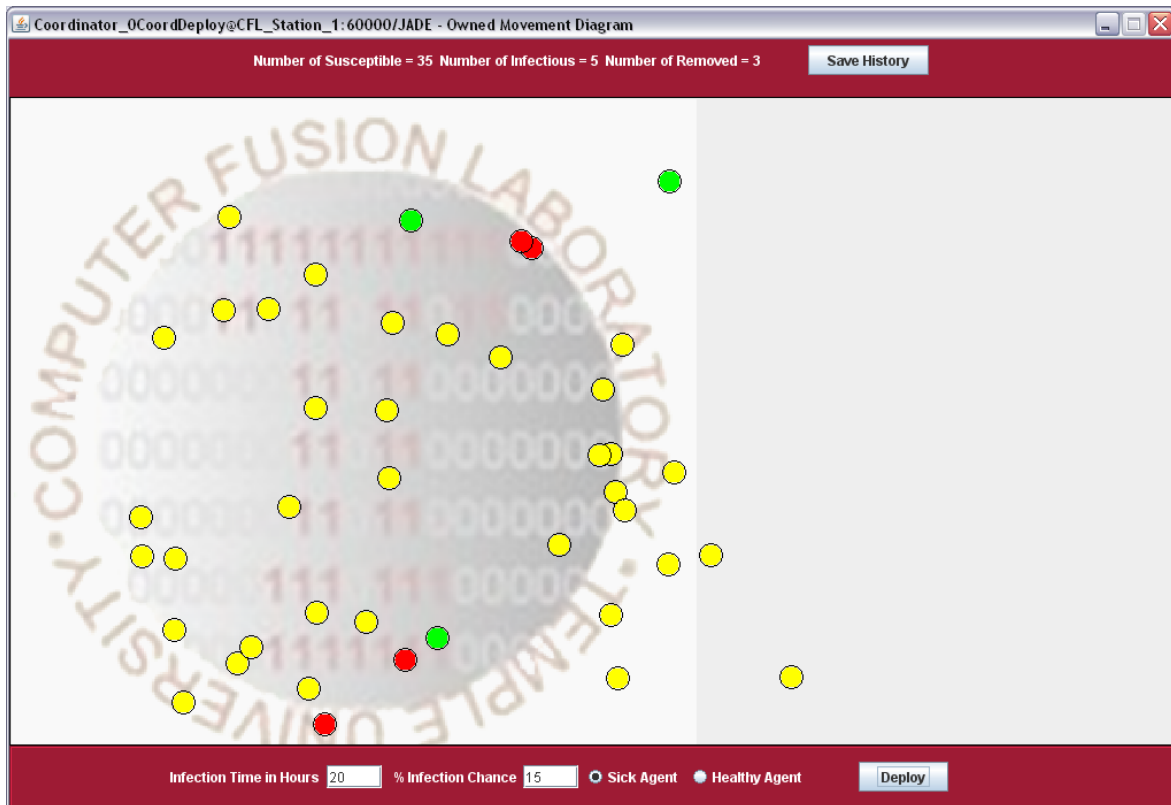


Figure 23: Application Front End Final Product

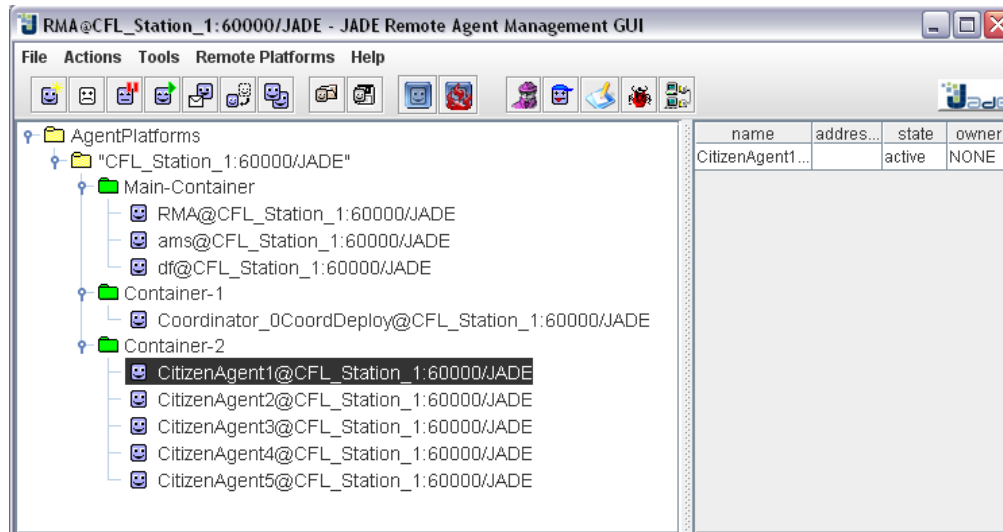


Figure 24: JADE Agent Management GUI

3. System Requirements

One of the major factors associated with our design is the dependency upon powerful computing equipment. JADE agents act in parallel, resulting in approximately 12-20 processor threads per agent, as well as threads for the agent management platform and GUI. Due to the multi-threading approach, a multi-core processor is recommend, as the application was developed on a Microsoft Windows® XP machine with an Intel® Core 2

Duo 3.0 GHz processor with 3Gb of RAM. Unfortunately these relatively high system requirements for the Application still result in a slow simulation. Due to the high complexity of messaging using the INGENIAS Conversation framework, agents are not able to process incoming data very quickly. To prevent the Coordinator agent from getting overloaded with incoming and outgoing sick messages, the simulation time step, that is, the time between agents moving from one location to the next and decrementing their counter, had to be increased to 3 seconds for fifty agents and 4 seconds for one hundred agents. This increases the simulation time to approximately three hours even on a more powerful machine. The Future Work section will reinforce that migrating away from the INGENIAS agent framework and hand-coding agent messaging structures would result in a faster system, as the JADE platform has been shown to efficiently handle a much higher message traffic load.

B. Analysis of SIR model

Despite the complex technical aspects involved in the design process, it is important to realize the primary objective of this project is to model the spread of disease in an arbitrary population. To gauge the viability of our approach, the SIR model provides an excellent benchmark for a successful epidemiology model. As previously discussed the SIR model can be considered as a Markov Chain. The transitional movement of agents between the states needs to be indexed with an appropriate time scale. Moreover, in order to verify our simulation's accuracy, an accepted SIR model was graphically plotted for verification. One of the benefits of our simulation and design is that it can model any arbitrary epidemic, given the correct epidemiology parameters in accordance with the SIR model. The specific case study known as the Hong Kong Flu of 1968 was used for the parameters of our simulation.

1. Hong Kong Flu SIR Simulation

In 1968 through 1969, Influenza A virus subtype H3N2 was an epidemic that spread worldwide, and is estimated to have killed nearly one million people (Viboud, Grais, Lafont, Miller, & Simonsen, 2005). The flu originated in Hong Kong and subsequently coined the title Hong Kong Flu. This specific case study was chosen primarily based on an abundance of available information. Figure 25 represents the desired SIR plot based on the assumption that the Hong Kong Flu parameters are correct. The plot was coded within MATLAB, using the *ode45* differential equation solver to produce discrete values based on the three differential equations. The initial conditions included an initial population of 7.9 million people, a run time of 100 days, and an initial infected population of one human.

Given the inherent weaknesses of the SIR model, such as large averages and assumptions, two disease parameters used in the simulation were sourced from hospital records in the case study. Therefore our infection rate was 56% (γ), with a maximum infectious period of 12 days. The initial population was 7.9 million people, however we scaled the population into percentages in order to possibly compare this "actual" plot with our simulation.

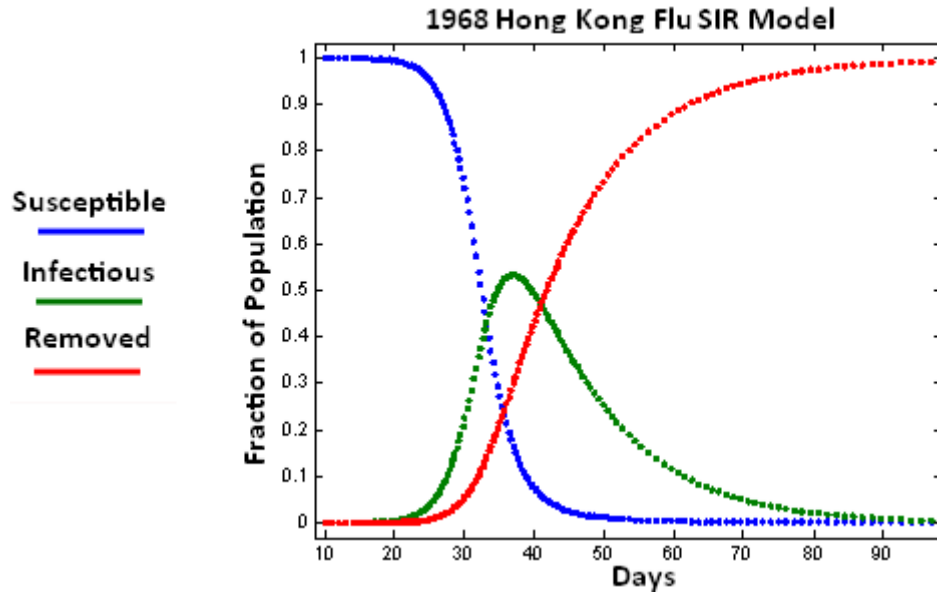


Figure 25: MATLAB Simulation of SIR Epidemiology Model

2. Simulation Using Senior Design Project

After completion of the final design, there were a few factors affecting the simulation, requiring an explanation of discrepancies between the simulation results and our accepted results. Due to time constraints and lack of computing power, the resolution and curvature of our plots as well as the time scaling, can help explain the disparity.

Perhaps the most obvious difference between the two plots is the low resolution from our simulation plot. This is due to the lack of agents deployed into the system, which were nearly 100 agents. If there were 7.9 million agents deployed then curve would have higher resolution given such a high deployment. We did not run a high deployment number primary due to lack of computing power, which is not to say the computer was not powerful enough, rather our product was a very computationally expensive simulation, intended to run across multiple computers.

Another large discrepancy between the two plots is the different time scale. Essentially this is a unit conversion issue that is difficult to correct within the multi-agent system code. Given the system is embedded with a discrete unit time step for tracking the interactions, the team decided to keep the graph units in time steps. This is so future developers may change the time step based on system specifications adjust their unit conversions afterwards. The unit step time used in this run was 3 seconds, because it was found to be an adequate length of time to finish all tasks before the next round of messaging. Shorter time steps resulted in delays and, in extreme cases, message buffer overflow as discussed earlier. A time conversion is simply multiplying 3 seconds by the number of steps (45) to produce the number of days, 135 – close to the SIR modeled time.

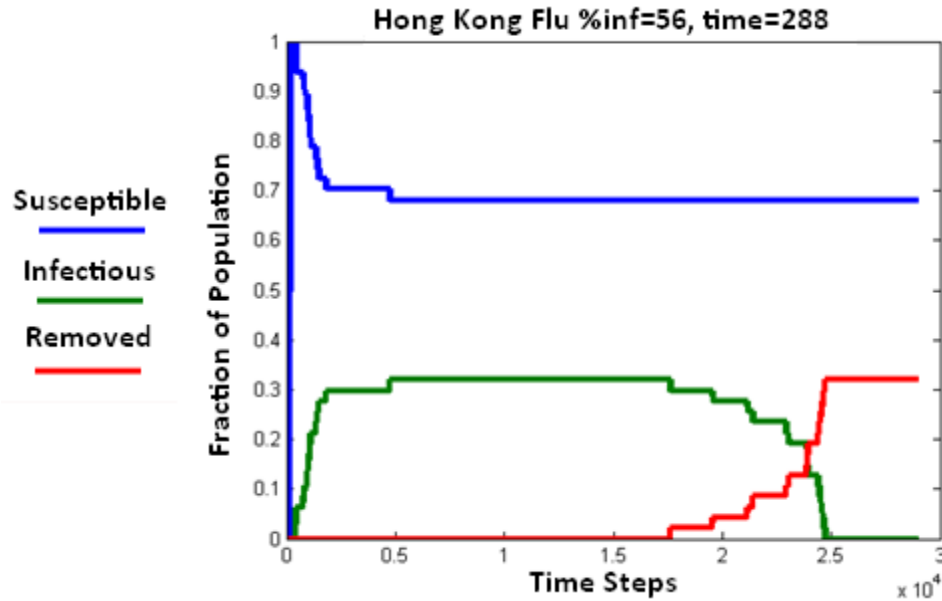


Figure 26: Senior Design Team Simulation Results

3. Analysis

The level of symmetry from the SIR plot can be seen graphically based on the mathematical equations. From the simulation plot, it can be inferred that our simulation results are acceptable and are a relative success, primarily due to the resemblance of the curved trend from the accepted plot. We can see that as the number of susceptible agents becomes infected through a rate of transmission (γ), the slope of the Removed curve is similar yet in the opposite direction. This intuitively makes sense because the total population size remains constant. After the epidemic has passed, the entire population should migrate into the removed sub-population. In addition, the slope of the Infectious curve is initially the same as the Removed curve. This ultimately makes sense, because the fundamental variable (β) is used in the differential equations for both curves. In addition, another important characteristic of the SIR plot is that the peak of the infectious curve occurs between the susceptible and infectious plots, and the plots reach a steady-state at around the same time (100-135 days). Also, in reality, the peak percentage of the population infected with Hong Kong Flu was much lower than 56%, lending the simulation results an added level of credibility over the SIR model.

V. Conclusions

The ultimate goal of this project was to demonstrate the validity and advantages of using a multi-agent system to simulate social situations. While the simulation was quite simple in its individual agent components, it managed to produce results close in accuracy to those of a proven mathematical model. The distributed computing model allows for even more complex environments and human characteristics in the future, such as a person going through intricate details of their day like deciding whether to take the subway or walk to work. While it had many drawbacks, INGENIAS did prove to expedite the development of multi-agent systems. An inexperienced computer engineer, under the

direction of an epidemiologist could expand upon the current model very quickly if needed. Overall, the project was a success, as all project goals were completed and results, in comparison to the SIR model, proved very promising.

VI. Acknowledgments

The senior design group would like to thank our advisors, Dr. Li Bai and Dr. Saroj Biswas, as well as Ethan Rublee of the Computer Fusion laboratory. They would also like to thank Jacquelin Speck for her technical writing assistance, Michael Korostelev for artistic design assistance, and Christopher Powell and Richard Hoffman for Java programming assistance.

VII. Recommendations for Future Work

A. Addition of Dynamic Behavioral Parameters

Perhaps one of the most compelling and interesting aspects of this project is the potential for future work. Additional work on this project can involve integrating a more complex human behavior. Our model simplified human behavior to traveling from home to the workplace. More advanced and detailed human models could be developed using the current software tools. Some examples are: different travel destinations, population grouping (schools, companies, families, etc.), and probabilistic decision making.

B. Multiple Platform Connectivity

From our final simulation results, it is quite evident that our SIR simulation plot is not completely reliable, primarily due to the small deployment. One of the major changes that can be made to improve the model is to run the simulation across multiple personal computers. When using multiple computers, the simulation could deploy more agents due to the increase in computational potential. Using the JADE Remote Management GUI, the senior design team successfully connected two JADE platforms on separate PCs. However the simulation demonstrated ran only on one computer due to time constraints.

Another fundamental change in the design process for this project would be to migrate away from using the INEGIAS platform entirely. From our experience using this platform, we concluded that despite the advantages in simplifying multi-agent design for developers, INGENIAS generates code which runs relatively slow due to a high degree of object creation and deletion. By using a different multi-agent platform, future developers could reduce system requirements and the simulation time from three hours.

VIII. References

- Dunham, J. B. (2005). An Agent-Based Spatially Explicit Epidemiological Model in MASON. *Journal of Artificial Societies and Social Simulation* , 9 (1), 10-12.
- FIPA00023. (2004, March). *FIPA- Agent Management Specification 00023*. (IEEE Foundation for Intelligent Physical Agents) Retrieved from <http://www.fipa.org/specs/fipa00023/SC00023K.pdf>

- FIPA00061. (2002, March). *FIPA- ACL Message Structure Specification 00061*. (IEEE Foundation for Intelligent Physical Agents) Retrieved from <http://www.fipa.org/specs/fipa00061/SC00061G.pdf>
- FIPA00067. (2002, March). *FIPA- Agent Message Transport Service 00067*. (IEEE Foundation for Intelligent Physical Agents) Retrieved from <http://www.fipa.org/specs/fipa00067/SC00067F.pdf>
- Gomez-Sanz, J. J. (2008). *GRASIA Development Guide Version 1.0*. Retrieved from <http://grasia.fdi.ucm.es/main/?q=en/print/book/export/html/140>
- Gomez-Sanz, J. J., & Gomez-Sanz, P. (2008, April). *GRASIA INGENIAS Development Kit (IDK) Manual*. Retrieved from <http://grasia.fdi.ucm.es/main/myfiles/ingeniasmanual.v14.pdf>
- Hawker, J., Begg, N., Blair, I., Reintjes, R., & Weinberg, J. (2005). *Communicable Disease Control Handbook, 2nd ed*. Wiley-Blackwell.
- IEEE Foundation for Intelligent Physical Agents. (2010). Retrieved from <http://www.fipa.org/>
- Nelson, K. E. (2005). In *Infectious Disease Epidemiology: Theory and Practice* (pp. 152-154). Jones & Bartlett Publishers, Inc.
- Nwana, H. S., Ndumu, D. T., Lee, L. C., & Collis, J. C. (2002, February). *ZEUS: A Toolkit and Approach for Building Distributed Multi-Agent System*.
- Olinick, M. (1978). *An Introduction to Mathematical Models in the Social Sciences*. Reading, Massachusetts: Addison-Wesley.
- Oracle Corporation. (2010). *What Is an Object?* Retrieved from The Java Tutorials: <http://java.sun.com/docs/books/tutorial/java/concepts/object.html>
- Tassier, T. (2005). *Fordham University*. Retrieved from <http://www.fordham.edu/images/Undergraduate/economics/faculty/SIR.pdf>
- Telecom Italia SpA. (2000, February). *Java Agent DEvelopment Framework (JADE)*. Retrieved from <http://jade.tilab.com/>
- Teweldemedhin, E., Marwala, T., & Mueller, C. (2004). Agent-based modelling: A Case Study in HIV Epidemic. *Fourth International Conference* (pp. 154-159). Johannesburg: School of Computer Science, School of Electrical and Information Engineering University of the Witwatersrand.
- University of Madrid. (2005). *GRASIA- INGENIAS*. Retrieved from <http://grasia.fdi.ucm.es/main/?q=en/node/128>
- Viboud, C., Grais, R., Lafont, B., Miller, M., & Simonsen, L. (2005). Multination Impact of the 1968 Hong Kong Influenza Pandemic: Evidence for a Smoldering Pandemic. *The Journal of Infectious Diseases* , 16.
- Wasserman, S., & Faust, K. (1994). *Social Network Analysis: Methods and Applications*. Cambridge University Press.
- Weiss, G. (1999). *Multi-Agent Systems- A Modern Approach to Distributed Artificial Intelligence*. MIT Press.

IX. Appendix A – Time Schedule

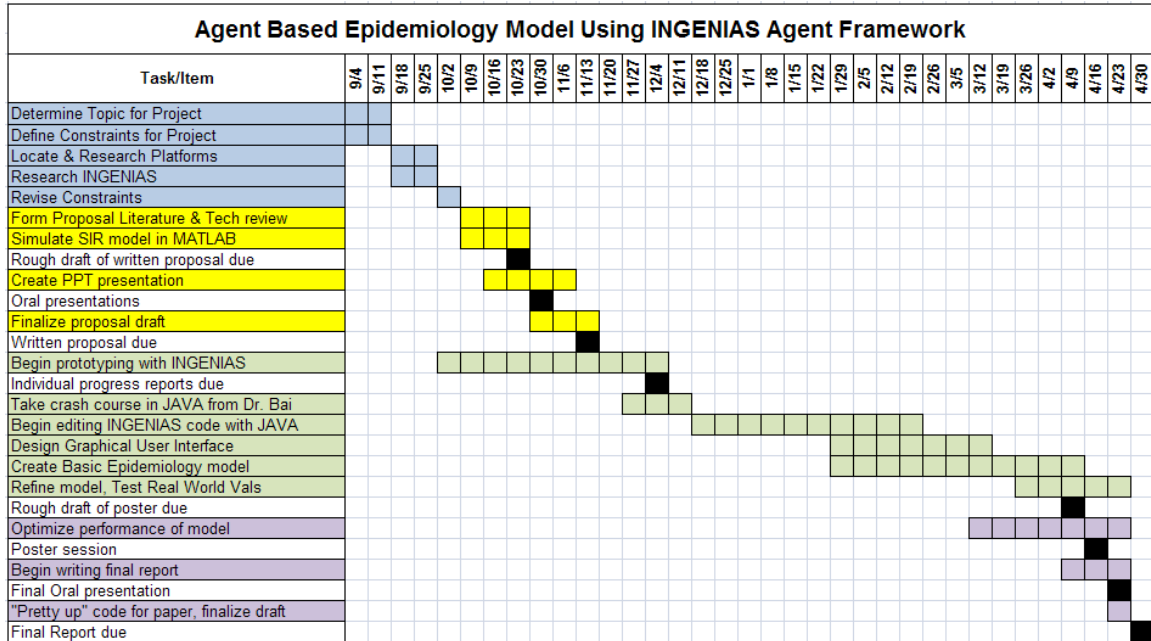


Table 1: Gantt chart illustrating deadlines for the project and intermediate tasks

X. Appendix B – Budget

All of the equipment needed for the project was available in-house. The team Faculty Advisor provided a development computer, negating the sole cost on the budget, shown in Table 1. Also, software licenses for MATLAB are made available to all College of Engineering students by Temple University. As per our design requirements, JADE is open-source software and is available for free (Telecom Italia SpA, 2000), the INGENIAS software from the Universidad Complutense Madrid (UCM) is also available for free through open-source licensing (University of Madrid, 2005). The table below summarizes the rest of the components which are all open software for this project yielding a zero dollar expense total.

Component	Quantity	Price
Windows Computer	1	\$400.00
JADE	1	\$0.00
INGENIAS IDK	1	\$0.00
Apache ANT	1	\$0.00
JAVA JDK	1	\$0.00
NetBeans IDE	1	\$0.00
Total		\$400.00
End Cost		\$0.00

Table 2: Project Budget. Open source software and donated computers round out a zero dollar budget

XI. Appendix C – Java Code

This code was developed by the senior design team, or modified from existing INGENIAS code.

A. RunSimulation.java

```
package ingenias.jade;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.net.Socket;
import java.net.UnknownHostException;

import jade.core.*;
import ingenias.jade.mental.*;

import ingenias.jade.graphics.MainInteractionManager;

public class RunSimulation {

    public static void main(String args[]) throws Exception{
        IAFProperties.setGraphicsOn(false);

        new Thread(){
            public void run(){
                String[] args1=new String[5];
                                args1[0]="-gui";

                args1[1]="-port";
                args1[2]="60000";
                args1[3]="-file-dir";
                args1[4]="jade/";
                jade.Boot.main(args1);
            }
        }.start();

        // Get a hold on JADE runtime
        jade.core.Runtime rt = jade.core.Runtime.instance();

        // Exit the JVM when there are no more containers around
        rt.setCloseVM(true);

        // Create a default profile
        Profile p = new ProfileImpl();
        p.setParameter("preload","a*");
        p.setParameter(Profile.MAIN_PORT, "60000");
        p.setParameter(Profile.FILE_DIR, "jade/");

        // Waits for JADE to start
        boolean notConnected=true;

        while (notConnected){
            try {
                Socket s=new
Socket("localhost",Integer.parseInt("60000"));

```

```

        notConnected=false;
    } catch (NumberFormatException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (UnknownHostException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IOException e) {

        System.err.println("Error: "+e.getMessage());
        System.err.println("Reconnecting in one second");
        try {
            Thread.currentThread().sleep(1000);
        } catch (InterruptedException e1) {
            // TODO Auto-generated catch block
            e1.printStackTrace();
        }
    }
}

// Create a new non-main container, connecting to the default
// main container (i.e. on this host, port 1099)
final jade.wrapper.AgentContainer ac =
rt.createAgentContainer(p);
{
    // Create a new agent
    final jade.wrapper.AgentController agcCoordinator_0CoordDeploy
= ac.createNewAgent("Coordinator_0CoordDeploy",
    "ingenias.jade.agents.CoordinatorJADEAgent", new
Object[0]);
    new Thread(){
        public void run(){
            try {
                agcCoordinator_0CoordDeploy.start();
            } catch (Exception e){
                e.printStackTrace();
            }
        }
    }.start();
}
}
}
}

```

B. LocAppAppImp.java

```

/**
 *
 * @author William R Moser, william.moser@temple.edu
 *
 * LocApp provides several utilities for agents, mostly generating
initial values for the agent
 */
package ingenias.jade.components;
import java.util.Random;

public class LocAppAppImp extends LocAppApp{

```

```

public LocAppAppImp() {
    super();
}
////////////////////////////////////
    Random rand = new Random();
    public boolean IsActive = false;
    public int count_dir = 1;
    public int move_index = 0;
    public static double DIST_MAX=100.0;
    public double number_of_moves = 30+(rand.nextDouble()*25);
    public double [] work1 = new double[2];
    public double [] home1 = new double[2];
    public double LIM_X = 700;
    public double LIM_Y= 600;
    public static double [] current_location = new double[2];

    public int getNumMoves () {
        return((int) number_of_moves);
    }

public double[] setCurrentLocation(double MyX, double MyY){
    current_location[0]=MyX;
    current_location[1]=MyY;
    return(current_location);
}

public boolean proxQuery(double HisX, double HisY){
    boolean is_close = closeEnough(current_location[0],
current_location[1], HisX, HisY, DIST_MAX);
    return(is_close);
}

public static boolean closeEnough(double x1, double y1, double x2,
double y2,double distMax){
    double distX=Math.abs(x2-x1);
    double distY=Math.abs(y2-y1);
    if(x1==x2 || y1==y2)
    {
        double dist=(distX+distY);

        return (dist<=distMax);

    } else {
        double dist=Math.sqrt(distX*distX + distY*distY);

        return (dist<=distMax);
    }
}

    public double[] randHome(){
        Random rand = new Random ();

        double[] Home={0,0};
        Home[0]= rand.nextDouble() * LIM_X;
        Home[1]= rand.nextDouble() * LIM_Y;
    }

```

```

        return(Home);
    }
    public double[] randWork(){
        Random rand = new Random ();
        double[] Work={0,0};
        Work[0]= rand.nextDouble() * LIM_X;
        Work[1]= rand.nextDouble() * LIM_Y;
        return(Work);
    }
    public double[][] moveHome2Work(double[] Work, double[] Home){
        double[][] intermed= new double[2][(int) number_of_moves];
        double dx=0;
        double dy=0;
        double norm= number_of_moves;
        intermed[0][0]=Home[0];
        intermed[1][0]=Home[1];
        double distX=Math.abs(Work[0]-Home[0]);
        double distY=Math.abs(Work[1]-Home[1]);
        if(Work[0]<Home[0])
        {
            dx=-1;
        }
        if(Work[0]>Home[0])
        {
            dx=1;
        }
        if(Work[1]<Home[1])
        {
            dy=-1;
        }
        if(Work[1]>Home[1])
        {
            dy=1;
        }

        for(int i = 1; i<=number_of_moves-1; i++)
        {
            intermed[0][i]=intermed[0][i-1]+(dx*distX/norm);
        }

        for(int j = 1; j<=number_of_moves-1; j++)
        {
            intermed[1][j]=intermed[1][j-1]+(dy*distY/norm);
        }

        return(intermed);
    }

    public void showLocation(){
        return;
    }
}

```

C. CoordAppAppImp.java

```
/**
 *
 * @author William R Moser, william.moser@temple.edu
 *
 * CoordApp provides several utilities for agents, mostly movement and
 * contracting illnesses
 */

package ingenias.jade.components;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.*;

import ingenias.jade.exception.*;
import ingenias.jade.mental.*;

public class CoordAppAppImp extends CoordAppApp{

    public CoordAppAppImp() {
        super();
    }
    Sickness sick = new Sickness();
    double[] location = new double[3];
    public static double DIST_MAX=30.0;

    public double[] getAInfo(){
        double [] CitInf = new double[3];
        CitInf[0]=location[0];
        CitInf[1]=location[1];
        CitInf[2]=sick.Status;
        return(CitInf);
    }

    public boolean proxQuery(double HisX, double HisY){
        boolean is_close = closeEnough(location[0], location[1], HisX,
        HisY, DIST_MAX);
        // System.out.println("Agent prox check is "+is_close);
        // System.out.println("because this agent is at "+location[0] +"," +
        +location[1] +" and the other is at " +HisX +"," +HisY);
        return(is_close);
    }
    public static boolean closeEnough(double x1, double y1, double x2,
    double y2, double distMax){
        double distX=Math.abs(x2-x1);
        double distY=Math.abs(y2-y1);
        if(x1==x2 || y1==y2)
        {
            double dist=(distX+distY);

            return (dist<=distMax);

        } else {
            double dist=Math.sqrt(distX*distX + distY*distY);
        }
    }
}
```

```

        return (dist<=distMax);
    }

}

public void setSick(int Sickness, double Resistance){
    sick.getSick(Sickness, Resistance);
}

public void agentMove(int AgentID, double[][] path, int
number_of_moves){
    final int AgentID1 = AgentID;
    final double[][] path1 = path;
    final int number_of_moves1 = number_of_moves;
    if(path==null || AgentID == 0){
        long t0, t1;
        t0 = System.currentTimeMillis();
        do{
            t1 = System.currentTimeMillis();
            System.out.println("Time Sink. . .");
        } while ((t1 - t0) < (500));
    }
    else if(path!=null && AgentID != 0){
        new Thread(){
            public void run(){
                try {

movIng (AgentID1,path1,number_of_moves1);
                } catch (InterruptedException ex) {
                    System.out.println("exception reported
in agentMove!");
                }
            }
        }.start();
    }
}

public void movIng(int AgentID, double[][] path, int
number_of_moves)throws InterruptedException{
    ingenias.jade.components.AgentWindowAppApp gui =
AgentWindowAppInit.getInstance();
    int move_index = 0;
    int stopcount = 0;
    int count_dir = 1;
    boolean IsActive=true;

    while(IsActive && path!=null && number_of_moves!=0){
        Thread.sleep(4000);
        if(move_index != (number_of_moves-1) && move_index != 0)
        {
            move_index=move_index+count_dir;
            location[0] = path[0][move_index];
            location[1] = path[1][move_index];
            gui.placeAgent(AgentID, path[0][move_index],
path[1][move_index], sick.Status);
            stopcount++;
        }
        else if(move_index == 0)

```

```

        {
            count_dir=1;
            move_index=move_index+count_dir;
            location[0] = path[0][move_index];
            location[1] = path[1][move_index];
            gui.placeAgent(AgentID, path[0][move_index],
path[1][move_index], sick.Status);
        }
        else if(move_index == (number_of_moves-1))
        {
            count_dir=-1;
            move_index=move_index+count_dir;
            location[0] = path[0][move_index];
            location[1] = path[1][move_index];
            gui.placeAgent(AgentID, path[0][move_index],
path[1][move_index], sick.Status);
        }
        if(sick.Status==2){
            DoWaitForSick nevent= new DoWaitForSick();
            try {
                getOwner().getMSM().addMentalEntity(nevent);
            } catch (ingenias.exception.InvalidEntity ex){
                ex.printStackTrace();
            }
        }
        //System.out.println(AgentID + " " +
(int)path[0][move_index] + " " + (int)path[1][move_index]);
    }
}
}

```

D. AgentWindowApplmp

```

/* This File Creates a Window to Display the agent movement.
 * it can also output the movements to a file.
 * @author William R Moser <william.moser@temple.edu>
 *
 */

```

```

package ingenias.jade.components;
import jade.wrapper.StaleProxyException;
import java.awt.*;
import java.awt.event.*;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.*;
import java.io.*;
import java.util.*;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.IOException;

import jade.core.*;

```

```

public class AgentWindowAppAppImp extends AgentWindowAppApp {
    public int agenttype = 1;
    public static Hashtable AgentArray = new Hashtable();
    public static Hashtable AgentCreate = new Hashtable();
    public static String filename = "C:/data.txt";
    public static FileOutputStream os = null;
    JFrame showAgentWindow = new JFrame();
    AgentPanel agentpanel;
    JPanel toppanel = new JPanel();
    JLabel msgL = new JLabel("");

    JPanel bottompanel = new JPanel();
    public static JTextField right1 = new JTextField("20");
    public static JTextField right2 = new JTextField("15");
    public static JTextField right3 = new JTextField("100");
    public static JTextField right4 = new JTextField("1");
    public static int agentID = 0;
    JLabel left1 = new JLabel("Infection Time in Hours");
    JLabel left2 = new JLabel(" % Infection Chance");
    JLabel left3 = new JLabel(" Simulation Time");
    JLabel left4 = new JLabel(" # of Agents");
    Component component3 = Box.createHorizontalStrut(10);
    Component component2 = Box.createHorizontalStrut(5);
    Component component4 = Box.createHorizontalStrut(5);
    JButton btn_stop = new JButton("Kill");

    // Stop Button
    JRadioButton radio_sick = new JRadioButton("Sick Agent");
    // Start Button
    JRadioButton radio_healthy = new JRadioButton("Healthy
Agent"); // Start Button
    ButtonGroup radio_group = new ButtonGroup();
    JButton btn_health = new JButton(" Deploy ");
    JPanel box_buttons = new JPanel();
    JPanel box_oldbuttons = new JPanel();
    final JFileChooser fc = new JFileChooser();
    // BottomPanel bottompanel;
    int grIndex = 1;
    int AgentID = 0;

    public AgentWindowAppAppImp() throws FileNotFoundException{
        super();
        os = new FileOutputStream(filename);
    }

    public int getID(){
        AgentID++;
        AgentArray.put("Agent"+(AgentID), new AgentDot());
        System.out.println("Agent"+(AgentID));
        return (AgentID);
    }
}

```

```

        public void placeAgent(int AgentNumber, double AgentX, double
AgentY, int State){
            //moved to previous method getID
            AgentDot ad = (AgentDot)
AgentArray.get("Agent"+(AgentNumber));
            if (ad != null){
                Random rand = new Random();
                ad.agentnumber = AgentNumber;
                ad.x = AgentX;
                ad.y = AgentY;
                ad.state = State;
                ad.initiated = true;
            }
        }

        public int[] getInputs(){
            int[] input = new int[4];
            String InfLen = right1.getText();
            String PercInf = right2.getText();
            String Time = right3.getText();
            input[0] = Integer.parseInt(InfLen);
            input[1] = Integer.parseInt(PercInf);
            input[2] = Integer.parseInt(Time);
            input[3] = agenttype;
            return(input);
        }

        public void showAgentWindow() {

            agentpanel = new AgentPanel();    //make a new JPanel

agentpanel.setBorder(BorderFactory.createLineBorder(Color.black));
            createTopPanel();
            createBottomPanel();
            bottompanel.setSize(1020, 100);

            Container pane = showAgentWindow.getContentPane();
//container pane?
            pane.add( agentpanel, BorderLayout.CENTER ); //add the pane
at the center of the window
            pane.add( toppanel, BorderLayout.NORTH);
            pane.add(bottompanel, BorderLayout.SOUTH);
            Timeren(); //wait 1000 ms before redrawing
            showAgentWindow.setSize(new Dimension(1020,700));
            showAgentWindow.setTitle(getOwner().getName()+" - Owned
Movement Diagram");
            showAgentWindow.setResizable(false);

showAgentWindow.setDefaultCloseOperation(showAgentWindow.EXIT_ON_CLOSE)
;
            showAgentWindow.doLayout();
            showAgentWindow.setVisible(true);
        }

        public void createTopPanel()

```

```

{
    toppanel = new JPanel();
    JButton save = new JButton("Save History");
    Color c = new Color(.6196f, .10588f, .2039f);
    toppanel.setBackground(c);
    msgL.setForeground(Color.white);
    toppanel.add(msgL);
    toppanel.add(save);
    fc.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);
    save.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            int returnVal =
fc.showSaveDialog(toppanel);
            if (returnVal ==
JFileChooser.APPROVE_OPTION) {
                File file = fc.getSelectedFile();
            }

        } //end actionPerformed
    }); //end actionlistener
    toppanel.setPreferredSize(new Dimension(50,50));

}

public void createBottomPanel()
{
    bottompanel = new JPanel();
    Color c = new Color(.6196f, .10588f, .2039f);
    bottompanel.setBackground(c);
    right1.setColumns(4);
    right2.setColumns(4);
    right3.setColumns(4);
    right4.setColumns(4);
    radio_sick.setBackground(c);
    radio_sick.setForeground(Color.white);
    radio_healthy.setBackground(c);
    radio_healthy.setForeground(Color.white);
    radio_healthy.setSelected(true);
    left1.setForeground(Color.white);
    left2.setForeground(Color.white);
    left3.setForeground(Color.white);
    left4.setForeground(Color.white);
    jade.core.Runtime rt = jade.core.Runtime.instance();
    rt.setCloseVM(true);
    Profile p = new ProfileImpl();
    p.setParameter("preload", "a*");
    p.setParameter(Profile.MAIN_PORT, "60000");
    p.setParameter(Profile.FILE_DIR, "jade/");

    radio_sick.addActionListener(new
java.awt.event.ActionListener() {
        public void actionPerformed(ActionEvent e) {
            agenttype = 2;
        }
    });
}

```

```

        radio_healthy.addActionListener(new
java.awt.event.ActionListener() {
    public void actionPerformed(ActionEvent e) {
        agenttype = 1;
    }
});
//Stop Button
btn_stop.setEnabled(false);
btn_stop.addActionListener(new
java.awt.event.ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("The Stop Button Action
Performed");
    }
});
//Start Button

    final jade.wrapper.AgentContainer ac =
rt.createAgentContainer(p);{
    btn_health.addActionListener(new
java.awt.event.ActionListener() {
        public void actionPerformed(ActionEvent e) {
            // Create a new agent
            final jade.wrapper.AgentController agcCitizenAgent;
            try {
                agentID++;
                agcCitizenAgent =
ac.createNewAgent("CitizenAgent"+agentID,
"ingenias.jade.agents.CitizenAgentJADEAgent", new Object[0]);
                new Thread(){
                    public void run(){
                        try {

                            agcCitizenAgent.start();

                        } catch (Exception e){
                            e.printStackTrace();
                        }
                    }
                }.start();
            } catch (StaleProxyException ex) {
                //
Logger.getLogger(BottomPanel.class.getName()).log(Level.SEVERE, null,
ex);
            }

        } //end wrapper
    }); //end action performed
}
bottompanel.add(left1);
bottompanel.add(right1);
bottompanel.add(left2);
bottompanel.add(right2);
//bottompanel.add(left3);
// bottompanel.add(right3);
radio_group.add(radio_sick);

```

```

        radio_group.add(radio_healthy);
        bottompanel.add(radio_sick);
        bottompanel.add(radio_healthy);
        //bottompanel.add(left4);
        // bottompanel.add(right4);
        bottompanel.setBorder( BorderFactory.createCompoundBorder(
BorderFactory.createEtchedBorder(), BorderFactory.createEmptyBorder( 2,
2, 2, 2 ) ) );
        box_oldbuttons.add(component2);
        box_oldbuttons.add(component3);
        // box_oldbuttons.add(btn_stop);          //Stop
        box_oldbuttons.add(component4);
        box_oldbuttons.add(btn_health);
        box_oldbuttons.setSize(700, 25);
        box_oldbuttons.setBackground(c);
        bottompanel.add(box_oldbuttons);
    }

    public void Timeren(){ //seems like this is an Enable Timer
        int delay = 50;      //1000 ms delay (1 sec)

        ActionListener taskPerformer = new ActionListener() { //Add
action listenter (still in Timeren)

            public void actionPerformed(ActionEvent evt) { //when
the task (evt) is done
                agentpanel.repaint(); //repaint the whole thing with
current positions
                int[] c = agentpanel.color;
                msgL.setText("Number of Susceptible = " +c[0] +"
Number of Infectious = " +c[1] +" Number of Removed = " +c[2] +"
");
                toppanel.repaint();
            }
        };
        new javax.swing.Timer(delay, taskPerformer).start(); //the
Timer repeats the redraw on a delay
    }

}

class AgentPanel extends JPanel {
    public int[] color = {0, 0, 0};
    public AgentPanel () {

        for ( int x = 0 ; x < AgentWindowAppAppImp.AgentArray.size()
; x++ ){ //set each element of the array to an agent
            AgentWindowAppAppImp.AgentArray.put("Agent"+(x+1), new
AgentDot());
        }

        setPreferredSize (new Dimension(600,600));
    }
}

```

```

        protected void paintComponent (Graphics agents) {
            super.paintComponent (agents);    //paint the thing fed to it
            final ImageIcon icon = new ImageIcon("img/bg.png");
            agents.drawImage(icon.getImage(), 0, 0, 600, 600, null);
            color[0]=0;
            color[1]=0;
            color[2]=0;
            Random rand = new Random();
            for ( int x = 0 ; x <
AgentWindowAppAppImp.AgentArray.size(); x++ ){
                //for all the dots, paint the with drawDots
                AgentDot ad = (AgentDot)
AgentWindowAppAppImp.AgentArray.get("Agent"+(x+1));
                if (ad != null){
                    if(ad.initiated==true){
                        ad.drawDots (agents);
                        if(ad.state==1){
                            color[0]++;
                        }
                        else if(ad.state==2){
                            color[1]++;
                        }
                        else if(ad.state==3){
                            color[2]++;
                        }
                    }

                    } //end chack for true
                    else if(ad.initiated==false){
                    }
                }
            }

            try {
                String tmp = color[0] + " " + color[1] + " " + color[2] +
"\r\n";
                AgentWindowAppAppImp.os.write(tmp.getBytes());
                AgentWindowAppAppImp.os.flush();
            } catch (IOException ex) {

Logger.getLogger (AgentPanel.class.getName()).log (Level.SEVERE, null,
ex);
        }
    }
}

```

E. AgentDot.java

```

/**
 * @author William R Moser
 *
 * Agent Dot draws the colored dots for the application
 */

package ingenias.jade.components;

```

```

import java.awt.Color;
import java.awt.Graphics;

public class AgentDot {
    private final int BALLDIAMETER = 20;
    public double x;
    public double y;
    public int state;
    public int agentnumber;
    public boolean initiated=false;

    public void drawDots( Graphics g ) { //pass graphics
        g.setColor( Color.BLACK ); //outside border
        g.drawOval((int) x, (int) y, BALLDIAMETER, BALLDIAMETER );
//draw at the postion
        if(state==1){
            g.setColor( Color.YELLOW );
        }
        else if(state==2){
            g.setColor( Color.RED );
        }
        else if(state==3){
            g.setColor( Color.GREEN );
        }
        g.fillOval( (int) x+1, (int) y+1, BALLDIAMETER-2,
BALLDIAMETER-2 );

    }
}

```

F. Sickness.java

```

//@author William R Moser, william.moser@temple.edu
//Sickness contains a counter to reduce the amount of time the agent is
sick for every time step

```

```

package ingenias.jade.components;
import java.awt.Toolkit;
import java.awt.Color;
import java.awt.Graphics;
import java.util.logging.Level;
import java.util.logging.Logger;

public class Sickness {
    public int Status = 1;
    public int time_sick = 0;
    public void getSick(int Sickness, double Resistance){

        final int Sickness1 = Sickness;
        final double Resistancel = Resistance;
        new Thread(){
            public void run(){
                try {

                    countdown(Sickness1, Resistancel);
                } catch (InterruptedException ex) {

```

```

        Logger.getLogger(Sickness.class.getName()).log(Level.SEVERE, null, ex);
    }
    }.start();
}

    public void countDown(int Sickness, double Resistance) throws
InterruptedException {
        time_sick = (int) (Sickness*Resistance);
        while(time_sick>=0){
            if(time_sick>=1){
                time_sick=time_sick-1;
                Status = 2; //susceptible
                Thread.sleep(13600);
            }
            if(time_sick == 0 && Status == 2){//if the sick timer runs
out, go to removed
                Status = 3; //removed
                Thread.yield();
            }

        } //end while(time_sick)
    }
}

```

G. CommuteTaskTask.java

All Task code is preceded with the following comment block:

```

/*
    Copyright (C) 2005 Jorge Gomez Sanz
    This file is part of INGENIAS Agent Framework, an agent
    infrastructure linked to the INGENIAS Development Kit, and available at
    http://grasia.fdi.ucm.es/ingenias or http://ingenias.sourceforge.net.
    INGENIAS Agent Framework is free software; you can redistribute it
    and/or modify it under the terms of the GNU General Public License as
    published by the Free Software Foundation; either version 2 of the
    License, or (at your option) any later version. INGENIAS Agent Framework
    is distributed in the hope that it will be useful, but WITHOUT ANY
    WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
    for more details. You should have received a copy of the GNU General
    Public License along with INGENIAS Agent Framework; if not, write to
    the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston,
    MA 02111-1307 USA
*/

package ingenias.jade.components;

import java.util.*;
import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

```

```

public class CommuteTaskTask extends Task{

    public CommuteTaskTask(String id){
        super(id,"CommuteTask");
    }

    public void execute() throws TaskException{
        try{
            execute2();
        }
        catch(InterruptedException e){
            System.out.println(e);
        }
    }
    public void execute2() throws InterruptedException{

        CoordCitConsume
        eiCoordCitConsume=(CoordCitConsume)this.getFirstInputOfType("CoordCitConsume");

        CitizenStatus
        eiCitizenStatus=(CitizenStatus)this.getFirstInputOfType("CitizenStatus");

        CoordAppApp
        eaCoordApp=(CoordAppApp)this.getApplication("CoordApp");

        LocAppApp eaLocApp=(LocAppApp)this.getApplication("LocApp");

        Vector<TaskOutput> outputs = this.getOutputs();
        TaskOutput defaultOutput= outputs.firstElement();
        TaskOutput outputsdefault=findOutputAlternative("default",
            outputs);
        YellowPages yp=null; // only available for initiators of
interactions

    //start_node:INGENIASCodeComponent5 <--- DO NOT REMOVE THIS

    // @author William R Moser <william.moser@temple.edu>
    //System.out.println("Commute Task started");
    int number_of_moves = eaLocApp.getNumMoves();
    double[][] path=eiCitizenStatus.getPath();
    int ID = eiCitizenStatus.getAgentID();
    eaCoordApp.agentMove(ID, path, number_of_moves);
    //System.out.println("commute Task Finished");

    //end_node:INGENIASCodeComponent5 <--- DO NOT REMOVE THIS

    }

}

```

H. GetSickTask.java

```

package ingenias.jade.components;

import java.util.*;

```

```

import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

public class CommuteTaskTask extends Task{

    public CommuteTaskTask(String id){
        super(id,"CommuteTask");
    }

    public void execute() throws TaskException{
        try{
            execute2();
        }
        catch(InterruptedException e){
            System.out.println(e);
        }
    }

    public void execute2() throws InterruptedException{

        CoordCitConsume
        eiCoordCitConsume=(CoordCitConsume)this.getFirstInputOfType("CoordCitConsume");

        CitizenStatus
        eiCitizenStatus=(CitizenStatus)this.getFirstInputOfType("CitizenStatus");

        CoordAppApp
        eaCoordApp=(CoordAppApp)this.getApplication("CoordApp");

        LocAppApp eaLocApp=(LocAppApp)this.getApplication("LocApp");

        Vector<TaskOutput> outputs = this.getOutputs();
        TaskOutput defaultOutput= outputs.firstElement();
        TaskOutput outputsdefault=findOutputAlternative("default",
            outputs);
        YellowPages yp=null; // only available for initiators of
interactions

        //start_node:INGENIASCodeComponent5 <--- DO NOT REMOVE THIS

        // @author William R Moser <william.moser@temple.edu>
        //System.out.println("Commute Task started");
        int number_of_moves = eaLocApp.getNumMoves();
        double[][] path=eiCitizenStatus.getPath();
        int ID = eiCitizenStatus.getAgentID();
        eaCoordApp.agentMove(ID, path, number_of_moves);
        //System.out.println("commute Task Finished");

        //end_node:INGENIASCodeComponent5 <--- DO NOT REMOVE THIS

    }

}

```

1. InitializeTaskTask.java

```
package ingenias.jade.components;

import java.util.*;
import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

public class InitializeTaskTask extends Task{

    public InitializeTaskTask(String id){
        super(id,"InitializeTask");
    }

    public void execute() throws TaskException{

        InitConsume
        eiInitConsume=(InitConsume)this.getFirstInputOfType("InitConsume");

        InitFrameFact
        eiInitFrameFact=(InitFrameFact)this.getFirstInputOfType("InitFrameFact"
        );

        CitizenStatus
        eiCitizenStatus=(CitizenStatus)this.getFirstInputOfType("CitizenStatus"
        );

        CoordAppApp
        eaCoordApp=(CoordAppApp)this.getApplication("CoordApp");

        LocAppApp eaLocApp=(LocAppApp)this.getApplication("LocApp");

        Vector<TaskOutput> outputs = this.getOutputs();
        TaskOutput defaultOutput= outputs.firstElement();
        TaskOutput
        outputsdefault=findOutputAlternative("default",
        outputs);

        CoordCitConsume outputsdefaultCoordCitConsume=
        (CoordCitConsume)
        outputsdefault.getEntityByType("CoordCitConsume");
        YellowPages yp=null; // only available for initiators of
        interactions

        //start_node:INGENIASCodeComponent4 <--- DO NOT REMOVE THIS

        //@author William R Moser <william.moser@temple.edu>
        Random rand = new Random();
        double home[] = eaLocApp.randHome();
        double work[] = eaLocApp.randWork();
        double resistance = (rand.nextDouble()*0.7)+0.3; //resistance level, can
        resist up to 20% of illness
        eiCitizenStatus.setSickness(0);
        eiCitizenStatus.setCitX(home[0]);
```

```

eiCitizenStatus.setCitY(home[1]);
eiCitizenStatus.setAgentResistance(resistance);
eiInitFrameFact.setHomeX(home[0]);
eiInitFrameFact.setHomeY(home[1]);
eiInitFrameFact.setWorkX(work[0]);
eiInitFrameFact.setWorkY(work[1]);
double[][] path = eaLocApp.moveHome2Work(home,work);
eiCitizenStatus.setPath(path);

ingenias.jade.components.AgentWindowAppApp gui =
AgentWindowAppInit.getInstance();
int ID = gui.getID();
int[] input = gui.getInputs();
eiCitizenStatus.setPercInfect(input[1]);
eiCitizenStatus.setLengthInfect(input[0]);
if(input[3]==2){
    eaCoordApp.setSick(input[0], resistance);
    eiCitizenStatus.setStatus(2);
}
if(input[3]==1)
    eiCitizenStatus.setStatus(1);

eiCitizenStatus.setAgentID(ID);
gui.placeAgent(ID, home[0], home[1],input[3]);

//#end_node:INGENIASCodeComponent4 <--- DO NOT REMOVE THIS

}

}

```

J. ReportSickTask.java

```

package ingenias.jade.components;
import java.util.*;
import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

public class ReportSickTask extends Task{

    public ReportSickTask(String id){
        super(id,"ReportSick");
    }

    public void execute() throws TaskException{

        IsSickFlag
eiIsSickFlag=(IsSickFlag)this.getFirstInputOfType("IsSickFlag");

        CitizenStatus
eiCitizenStatus=(CitizenStatus)this.getFirstInputOfType("CitizenStatus"
);
    }
}

```

```

        // This means that the task participates in the interaction
CoordCitInteraction
        RuntimeConversation
conversationContextCoordCitInteraction=this.getConversationContext();

        Vector<TaskOutput> outputs = this.getOutputs();
        TaskOutput defaultOutput= outputs.firstElement();

        TaskOutput outputsdefault=findOutputAlternative("default",
                outputs);
        ReportFF outputsdefaultReportFF=
            (ReportFF)
                outputsdefault.getEntityByType("ReportFF");

        YellowPages yp=null; // only available for initiators of
interactions
//#start_node:INGENIASCodeComponent2 <--- DO NOT REMOVE THIS

//@author William R Moser, william.moser@temple.edu
//System.out.println("Report Task Start");
double SX = eiIsSickFlag.getSickX();
double SY = eiIsSickFlag.getSickY();
int ID = eiIsSickFlag.getAgentID();
double PercInfect = eiCitizenStatus.getPercInfect();
double LengthInfect = eiCitizenStatus.getLengthInfect();
outputsdefaultReportFF.setSickX(SX);
outputsdefaultReportFF.setSickY(SY);
outputsdefaultReportFF.setAgentID(ID);
outputsdefaultReportFF.setPercInfect((int)PercInfect);
outputsdefaultReportFF.setLengthInfect(LengthInfect);
//System.out.println("Report Task Finished");

//#end_node:INGENIASCodeComponent2 <--- DO NOT REMOVE THIS

    }

}

```

K. SendSickLocTaskTask.java

```

package ingenias.jade.components;
import java.util.*;
import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

public class SendSickLocTaskTask extends Task{

    public SendSickLocTaskTask(String id){
        super(id,"SendSickLocTask");
    }

    public void execute() throws TaskException{

```

```

        SickLocFF
eiSickLocFF=(SickLocFF)this.getFirstInputOfType("SickLocFF");

        // This means that the task participates in the interaction
SendInfect
        RuntimeConversation
conversationContextSendInfect=this.getConversationContext();

        Vector<TaskOutput> outputs = this.getOutputs();
        TaskOutput defaultOutput= outputs.firstElement();

        TaskOutput outputsdefault=findOutputAlternative("default",
        outputs);

        SendSickLocFF outputsdefaultSendSickLocFF=
        (SendSickLocFF)
        outputsdefault.getEntityByType("SendSickLocFF");

        YellowPages yp=null; // only available for initiators of
interactions

//#start_node:INGENIASCodeComponent6 <--- DO NOT REMOVE THIS

//@author William R Moser, william.moser@temple.edu
//System.out.println("SendLocTask Start");
double SX = eiSickLocFF.getSickX();
double SY = eiSickLocFF.getSickY();
int ID = eiSickLocFF.getAgentID();
double PercInfect = eiSickLocFF.getPercInfect();
double LengthInfect = eiSickLocFF.getLengthInfect();
outputsdefaultSendSickLocFF.setSickX(SX);
outputsdefaultSendSickLocFF.setSickY(SY);
outputsdefaultSendSickLocFF.setAgentID(ID);
outputsdefaultSendSickLocFF.setPercInfect((int)PercInfect);
outputsdefaultSendSickLocFF.setLengthInfect(LengthInfect);
//System.out.println("Send Task Finished");

//#end_node:INGENIASCodeComponent6 <--- DO NOT REMOVE THIS

    }

}

```

L. TestTaskTask.java

```

package ingenias.jade.components;
import java.util.*;
import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

public class TestTaskTask extends Task{

    public TestTaskTask(String id){
        super(id,"TestTask");
    }
}

```

```

public void execute() throws TaskException{

    ReportFF
    eiReportFF=(ReportFF)this.getFirstInputOfType("ReportFF");

    // This means that the task participates in the interaction
    CoordCitInteraction
    RuntimeConversation
    conversationContextCoordCitInteraction=this.getConversationContext();

    Vector<TaskOutput> outputs = this.getOutputs();
    TaskOutput defaultOutput= outputs.firstElement();

    TaskOutput outputsdefault=findOutputAlternative("default",
        outputs);

    RuntimeConversation outputsdefaultSendInfect=
        (RuntimeConversation)
        outputsdefault.getEntityByType("SendInfect");

    SickLocFF outputsdefaultSickLocFF=
        (SickLocFF)
        outputsdefault.getEntityByType("SickLocFF");

    YellowPages yp=null; // only available for initiators of
    interactions

    // This task can produce an interaction of type SendInfect by
    working with its conversation object
    // To define manually who are the collaborator involved. Your
    selection will be verified
    // in runtime. Pay attention to log messages to detect errors.
    You can use the yello pages
    // service to locate other agents
    yp=(YellowPages)this.getApplication("YellowPages");

    // Uncomment the following and write down a proper local id of the
    agent
    // Find an agent playing the role "CitRole"
    //eoSendInfect.addCollaborators("Local ID of the
    collaborator");

    //start_node:INGENIASCodeComponent3 <--- DO NOT REMOVE THIS

    //@author William R Moser, william.moser@temple.edu
    double SX = 0;
    double SY = 0;
    int ID = 0;
    double PercInfect = 0;
    double LengthInfect = 0;
    SX = eiReportFF.getSickX();
    SY = eiReportFF.getSickY();
    ID = eiReportFF.getAgentID();
    PercInfect = eiReportFF.getPercInfect();
    LengthInfect = eiReportFF.getLengthInfect();
    outputsdefaultSickLocFF.setSickX(SX);

```

```

outputsdefaultSickLocFF.setSickY(SY);
outputsdefaultSickLocFF.setAgentID(ID);
outputsdefaultSickLocFF.setPercInfect(PercInfect);
outputsdefaultSickLocFF.setLengthInfect(LengthInfect);
//System.out.println("Convo dump" +ID +", " +SX +", " +SY);
//#end_node:INGENIASCodeComponent3 <--- DO NOT REMOVE THIS

}

}

```

M. WaitForSickTask.java

```

package ingenias.jade.components;
import java.util.*;
import ingenias.jade.exception.*;
import ingenias.jade.comm.*;
import ingenias.jade.mental.*;
import ingenias.editor.entities.*;

public class WaitForSickTask extends Task{

    public WaitForSickTask(String id){
        super(id,"WaitForSick");
    }

    public void execute() throws TaskException{
        try{
            execute2();
        }
        catch(InterruptedException e){
            System.out.println(e);
        }
    }

    public void execute2() throws InterruptedException{

        DoWaitForSick
        eiDoWaitForSick=(DoWaitForSick)this.getFirstInputOfType("DoWaitForSick"
        );

        CitizenStatus
        eiCitizenStatus=(CitizenStatus)this.getFirstInputOfType("CitizenStatus"
        );

        CoordAppApp
        eaCoordApp=(CoordAppApp)this.getApplication("CoordApp");

        Vector<TaskOutput> outputs = this.getOutputs();
        TaskOutput defaultOutput= outputs.firstElement();

        TaskOutput outputsdefault=findOutputAlternative("default",
        outputs);

        RuntimeConversation outputsdefaultCoordCitInteraction=
        (RuntimeConversation)
        outputsdefault.getEntityByType("CoordCitInteraction");
    }
}

```

```

        IsSickFlag outputsdefaultIsSickFlag=
            (IsSickFlag)
            outputsdefault.getEntityByType("IsSickFlag");

        YellowPages yp=null; // only available for initiators of
interactions

        // This task can produce an interaction of type
CoordCitInteraction by working with its conversation object

        // To define manually who are the collaborator involved. Your
selection will be verified
        // in runtime. Pay attention to log messages to detect errors.
You can use the yello pages
        // service to locate other agents
        yp=(YellowPages)this.getApplication("YellowPages");
        // Uncomment the following and write down a proper local id of
the agent
        // Find an agent playing the role "CoordRole"
        //eoCoordCitInteraction.addCollaborators("Local ID of the
collaborator");

//#start_node:INGENIASCodeComponent1 <--- DO NOT REMOVE THIS

//@author William R Moser, william.moser@temple.edu
//System.out.println("Wait Task Started");
double[] status = new double[3];
double SX = 0;
double SY = 0;
int ID = eiCitizenStatus.getAgentID();
do{
    status = eaCoordApp.getAInfo();
    SX = status[0];
    SY = status[1];
    ID = eiCitizenStatus.getAgentID();
}while(status[2]==1 || SX == 0 || SY == 0);
if(status[2]==2){
    outputsdefaultIsSickFlag.setSickX(SX);
    outputsdefaultIsSickFlag.setSickY(SY);
    outputsdefaultIsSickFlag.setAgentID(ID);
    //System.out.println("Wait For sick output is "+SX +" " +SY +" "
+ID +" " +status[2]);
}
//System.out.println("Wait Task Finished");
//#end_node:INGENIASCodeComponent1 <--- DO NOT REMOVE THIS

}
}

```

XII. Resumes

William Moser is a senior in Electrical Engineering at Temple University. At Temple, William is a long time member and current president of the IEEE student chapter. He has also worked since 2009 at the Computer Fusion Laboratory at Temple University (<http://cflwww.eng.temple.edu>), a lab focused on computer communication and information processing. At CFL, William has worked on virtual 3D modeling and computer control schemes. After graduation, William plans to pursue a Masters in Electrical Engineering and eventually find work in the field of data security and wireless networks.

Bilal Malik is a senior in Electrical Engineering at Temple University. He is a member of the IEEE student branch at Temple University and director of public relations for the Pakistani Students Association. In addition, during the 2008 presidential election year, Bilal worked as an intern for the Barrack Obama campaign and served as director of canvassing for the north Philadelphia region. Bilal's chief interests include signal processing, control systems, and communications. After graduation, Bilal plans to continue his education with a Masters in Electrical Engineering in order to better prepare him for a career in controls system engineering.